Detection Of Hepatitis Virus Infection On Liver Biopsy Image Using Deep Learning

A Main project Submitted in Partial fulfilment of the Requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

Submitted by

GONABOINA HARIKA

Regd. No : 23F01F0021

Under the guidance of

Dr P.S.NAVEEN KUMAR. Ph.D

Associate Professor

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

ST. ANN'S COLLEGE OF ENGINEERING & TECHNOLOGY

AUTONOMOUS

[Approved by A.I.C.T.E. & Permanently Affiliated to JNTUK- Kakinada]

[Accredited by NBA (Four Times) and Permanently accredited by IE[I]]

[Accredited by NAAC with 'A' Grade (1st, 2nd and 3rd cycles) & Awarded 'A' Grade by AP State Govt.]

[Recognized by UGC u/s 2(f) & 12(B) of the UGC Act 1956]

Chirala – 523187, Bapatla District. URL: **www. sacet.ac.in**

Detection of Hepatitis Virus Infection_Document.doc

by DrMk .

Submission date: 14-May-2025 12:18PM (UTC+0700)
Submission ID: 2675478356
File name: Detection_of_Hepatitis_Virus_Infection_Document.doc (4.65M)
Word count: 13180
Character count: 81

Detection of Hepatitis Virus Infection_Document.doc

ORIGINALITY REPORT

8%	6%	8%	4%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	www.coursehero.com Internet Source	4%
2	Chams, V.. "Le virus GB-C ou virus << dit >> de l'hepatite G est-il implique en pathologie humaine ?", Transfusion clinique et biologique, 200309 Publication	2%
3	eitca.org Internet Source	2%

Exclude quotes	On	Exclude matches	< 2%
Exclude bibliography	On		

ST. ANN'S COLLEGE OF ENGINEERING & TECHNOLOGY

AUTONOMOUS

CHIRALA

DEPARTMENT OF MASTER OF COMPUTER APPLICATIONS

CERTIFICATE

This is to certify that Mr./Ms. **GONABOINA HARIKA** a final year MCA student of this college has successfully completed the Project work titled **Detection Of Hepatitis Virus Infection On Liver Biopsy Image Using Deep Learning** from January 2025 to May 2025, which has been submitted in partial fulfillment for the award of MCA degree from **Jawaharlal Nehru Technological University Kakinada, Kakinada.**

REGD. NO. : 23F01F0021

Internal Guide **Head of the Department**

Dr P.S Naveen Kumar

External Examiner

DECLARATION

I hereby declare that the report titled " **Detection Of Hepatitis Virus Infection On Liver Biopsy Image Using Deep Learning**" was completed during the period between January 2025 and May 2025, which has been submitted in partial fulfillment for the Award of MCA degree in St. Ann's College of Engineering & Technology from Jawaharlal Nehru Technological University-Kakinada, Kakinada.

I also declare that the implementation of this project is a result of my own effort and that it has not been submitted to any other university for the award of any degree or diploma.

Gonaboina Harika

(23F01F0021)

AKNOWLEDGMENT

The successful completion of any task is not possible without proper suggestion, guidance and environment. Combination of these three factors acts like backbone to my project titled **"Detection Of Hepatitis Infection On Liver Biopsy Image Using Deep Learning"**.

I express my sincere thanks to my guide **Dr. P. S. Naveen Kumar** , Head of the Department of Master Of Computer Applications, for his valuable suggestions during my course period, timely help, guidance and providing me with the most essential materials required for the completion of this work.

I am thankful to all teaching and non-teaching staff of the Master Of Computer Applications for the cooperation given for the successful completion of my project.

I would like to thank our Principal, **Dr. K. Jagadeesh Babu** for providing support and simulating environment. I would like to express my gratitude to the **Management** of **St. Ann's College of Engineering & Technology** for providing us a pleasant environment and excellent laboratory facilities.

Gonaboina Harika

(23F01F0021)

TABLE OF CONTENTS

	PAGE NO
PLAGIARISM	
CERTIFICATE	**I**
DECLARATION	**II**
ACKNOWLEDGEMENT	**III**
TABLE OF CONTENTS	**IV-VI**
LIST OF FIGURES	**VII**
LIST OF TABLES & ACRONYMS	**VIII**
ABSTRACT	**IX**
CHAPTER 1: INTRODUCTION……. 1-2	
1.1 PROBLEM STATEMENT……..…….. **3**	
1.2 EXISTING SYSTEM……..…….. **4-5**	
1.3 PROPOSED SYSTEM…….. **6**	
1.4 ORGANIZATION OF THESIS……. **8**	
CHAPTER 2: LITERATURE SURVEY…… 9-10	
2.1 BASIC CONCEPTS…….. **11-13**	
2.2 PROJECT RELATED WORK……... **14**	
CHAPTER 3: FEASIBILITY STUDY……. 15	
3.1 TECHNICAL FEASIBILIT……... **15**	
3.2 OPERATIONAL FEASIBILITY…… **15**	
3.3 ECONOMIC FEASIBILITY…….. **15**	

CHAPTER 4: SOFTWARE REQUIREMENT SPECIFICATIONS……... 16

4.1 FUNCTIONAL REQUIREMENTS……... **17**

4.2 NON-FUNCTIONAL REQUIREMENTS……. **18-21**

4.2.1 PERFORMANCE REQUIREMENTS……. **19**

4.2.2 HARDWARE REQUIREMENT…….. **19**

4.2.3 SOFTWARE REQUIREMENTS……. **20**

4.2.3.1 INTRODUCTION TO PYTHON…….. **21**

CHAPTER 5: SYSTEM MODELING…… 22

5.1 REQUIREMNETS MODELING……... **22-31**

5.1.1 CLASS DIAGRAM……. **22**

5.1.2 USE CASE DIAGRAM……... **23**

5.1.3 SEQUENCE DIAGRAM……. **25**

5.1.4 ACTIVITY DIAGRAM……... **27**

5.1.5 DATA FLOW DIAGRAM…… **28**

5.1.6 DEPLOYMENT DIAGRAM…….. **29**

5.1.7 MODULES DESCRIPTION……**30-31**

5.2 DESIGN MODELING…… **32-36**

5.2.1 SYSTEM ARCHITECTURE…….. **36**

CHAPTER 6: SYSTEM IMPLEMENTATION……. 37

6.1 DEVELOPMENT ENVIRONMENT AND TOOLS…… **37-39**

6.2 CODING……..……..…. **40-46**

6.2.1 SOURCE CODE……... **40-46**

CHAPTER 7: TESTING…… 47

7.1 INTRODUCTION TO TESTING……... **47-52**

7.1.1 TESTING STRATEGIES……. **48-49**

7.1.2 UNIT TESTING…… **50**

7.1.3 INTEGRATION TESTING…….. **50**

7.1.4 SYSTEM TESTING……. **50**

7.1.5 TEST CASES……... **51-52**

CHAPTER 8: RESULTS…… 53-59

8.1 SCREENS AND REPORTS……... **60-61**

8.2 USER MANUAL…… **62-63**

CHAPTER 9: LIMITATIONS……... 64

CHAPTER 10: CONCLUSION & FUTURE WORK…….. 65

CHAPTER 11: REFERENCES…….. 66

FORMATS

LIST OF FIGURES

FIG. NUM	FIG. NAME	PAGE NO
2.2.1	Sample image from DataSet	**14**
5.1.1	Class Diagram	**22**
5.1.2	Use Case Diagram	**23**
5.1.3	Sequence Diagram	**25**
5.1.4	Collaboration Diagram	**26**
5.1.5	Activity Diagram	**27**
5.1.6	Dataflow Diagram	**28**
5.1.7	Deployment Diagram	**29**
5.2.1	System Architecture	**36**
7.1.5.1	Test Case 1	**51**
7.1.5.2	Test Case 2	**51**
7.1.5.3	Test Case 3	**51**
7.1.5.4	Test Case 4	**52**
7.1.5.5	Test Case 5	**52**
7.1.5.6	Test Case 6	**52**
8.1	TensorFlow and GPU	**53**
8.2	Data Shape	**53**
8.3	Model Architecture	**54**
8.4	Training Metrics	**55**
8.5	Accuracy and Loss Graphs	**56**
8.6	Classification Report & Confusion Matrix	**57**
8.7	ROC Curve & AUC Score	**58**
8.8	Virus Detection Predictions	**59**
8.1.1	Screens & Report	**60**
8.1.2	Screens & Report	**60**
8.1.3	Screens & Report	**61**

LIST OF TABLES

TABLE NUMBER	TABLE NAME	PAGE NUMBER
2.1	Literature Review	**9 - 10**

LIST OF ACRONYMS

SHORT FORM	EXPANSION FORM
CNN	Convolutional Neural Networks
RNN	Recurrent Neural Networks
LSTM	Long Short-Term Memory
SVM	Support Vector Machine
AUC	Area Under the Curve
ROC	Receiver Operating Characteristic

ABSTRACT

Liver diseases, including conditions like hepatitis, cirrhosis, and liver cancer, pose significant health challenges that require timely and accurate diagnosis for effective treatment. Traditional diagnostic methods, such as CT scans and liver function tests, can be slow and prone to inaccuracies. In this study, we propose a deep learning-based approach that combines Convolutional Neural Networks (CNN) for image analysis and Long Short-Term Memory (LSTM) networks for handling sequential patient data. The hybrid CNN+LSTM model is designed to enhance the diagnosis and prognosis of liver diseases by effectively capturing both spatial and temporal features. Our results show that the CNN+LSTM model achieves exceptional performance, with an accuracy of 99%, perfect precision and recall (1.00 for both classes), and an Area Under the Curve (AUC) of 99%. These findings demonstrate that the CNN+LSTM model provides a highly reliable and accurate tool for liver disease diagnosis, offering significant improvements over traditional methods and aiding in better clinical decision-making.

CHAPTER 1

INTRODUCTION

Liver diseases, encompassing a spectrum of conditions from hepatitis and cirrhosis to hepatocellular carcinoma (HCC), represent a substantial and escalating global health challenge. These diseases often progress silently, with symptoms manifesting only in advanced stages, making early and accurate diagnosis critical for effective intervention and improved patient outcomes. Traditional diagnostic methods, while foundational, frequently encounter limitations that hinder their efficacy. These limitations include inherent subjectivity in interpretation, time-consuming procedural requirements, and difficulties in detecting subtle, early-stage pathological changes.

Conventional diagnostic approaches, such as serological tests, imaging techniques like computed tomography (CT) and ultrasound, and the histopathological evaluation of liver biopsy samples, form the cornerstone of current clinical practice. However, the manual analysis of liver biopsies, considered the gold standard, is both labor-intensive and prone to inter-observer variability, leading to inconsistencies and delays in diagnosis. Imaging techniques, while non-invasive, may lack the sensitivity to detect early-stage changes, and serological tests often provide indirect evidence of liver dysfunction.

The advent of digital pathology, enabling the capture of high-resolution images of biopsy slides, has opened new avenues for computer-aided diagnostic methods. Coupled with recent advancements in artificial intelligence, particularly deep learning, there is significant potential to transform medical diagnostics. Convolutional Neural Networks (CNN's) have demonstrated remarkable capabilities in analyzing complex image data, automatically extracting and learning hierarchical features directly from raw images. However, CNN's alone do not inherently account for the temporal dynamics of disease progression, which can provide crucial insights.

To address this limitation, Long Short-Term Memory (LSTM) networks have emerged as a powerful tool for modeling temporal dependencies in sequential data. By integrating CNN's and LSTMs into a hybrid model, we can leverage the strengths of both architectures, enabling robust feature extraction from images while simultaneously considering the temporal progression of disease indicators.

This document details the development and validation of a novel hybrid deep learning system designed to improve the detection of liver diseases, specifically hepatitis, through the integration of spatial and temporal data. The system utilizes CNN's to analyze high-resolution liver biopsy images and LSTMs to analyze sequential clinical data, such as laboratory test results and patient medical history. By combining these modalities, we aim to overcome the shortcomings of existing diagnostic practices and provide a more accurate, automated, and timely diagnostic tool.

The documentation outlines the problem domain, detailing the clinical significance and limitations of current diagnostic methods. It then presents the proposed methodology, including the architecture of the hybrid CNN+LSTM model, the dataset used for training and validation, and the evaluation metrics employed. The experimental results, demonstrating the performance of the hybrid model, are presented and compared with traditional diagnostic methods and standalone deep learning models. Finally, the document discusses the clinical implications of this research, highlighting the potential for improved patient outcomes and more effective clinical interventions.

The goal of this research is to demonstrate the potential of deep learning to revolutionize liver disease diagnosis, paving the way for more precise, data-driven medicine. This document serves as a comprehensive guide to the project, providing a detailed account of the methodology, results, and implications of this innovative approach.

1.1 PROBLEM STATEMENT

The current diagnostic methods for liver diseases, including hepatitis, are often limited by:

- **Subjectivity:** Manual analysis of liver biopsies is prone to inter-observer variability, leading to inconsistent diagnoses.
- **Time Consumption:** Traditional methods, especially histopathological examination, are labor-intensive and time-consuming.
- **Invasiveness and Patient Burden:** Minimizing the need for invasive diagnostic procedures, such as biopsies, by leveraging non-invasive imaging analysis enhanced with temporal patient data to reduce patient discomfort and potential complications.
- **Integration of Multi-Modal Data:** Effectively integrating and analyzing diverse patient data, including medical imaging (e.g., CT scans) and longitudinal clinical information (e.g., lab results, patient history), to provide a more holistic and accurate assessment of liver health.
- **Early-Stage Detection Challenges:** Conventional imaging and serological tests may fail to detect subtle, early-stage pathological changes.
- **Lack of Temporal Data Integration:** Existing methods primarily focus on static image analysis and do not effectively integrate temporal clinical data, which can provide crucial insights into disease progression.
- **Need for Automated and Accurate Diagnosis:** The need for a rapid, automated, and highly accurate diagnostic tool to improve clinical decision-making and patient outcomes, particularly in resource-limited settings.

1.2 EXISTING SYSTEM

Liver diseases, including hepatitis, cirrhosis, and liver cancer, represent a significant global health burden. Current diagnostic practices typically involve a combination of:

- **Clinical Assessment:** Patient history, physical examination, and symptom evaluation.
- **Serological Tests:** Blood tests to assess liver function and detect viral markers.
- **Imaging Techniques (CT, MRI, Ultrasound):** Visual assessment of liver structure and detection of abnormalities. These methods aid in identifying lesions, fibrosis, and other structural changes.
- **Liver Biopsy:** Histopathological examination of liver tissue, considered the gold standard for definitive diagnosis. This involves invasive procedures to obtain tissue samples.
- **Traditional Machine Learning (ML) and Deep Learning (DL) Applications:** Research efforts have explored ML/DL techniques for image analysis, biomarker analysis, and disease prediction.

Disadvantages of Existing Systems:

1. Subjectivity and Inter-observer Variability:

- ✧ Manual interpretation of imaging and biopsy results is prone to variations between clinicians, leading to inconsistent diagnoses.
- ✧ This subjectivity can significantly impact diagnostic accuracy and treatment decisions.

2. Time-Consuming Procedures:

- ✧ Histopathological examination of biopsies is labor-intensive and time-consuming, delaying diagnosis and treatment.
- ✧ Imaging analysis can also be time-consuming, especially for complex cases.

3. Invasiveness and Associated Risks:

- ✧ Liver biopsy is an invasive procedure with potential complications, including bleeding, infection, and pain.
- ✧ This limits its suitability for routine screening and repeated assessments.

4. Limited Sensitivity in Early-Stage Detection:

- ✧ Imaging techniques may not detect subtle, early-stage pathological changes, hindering timely intervention.
- ✧ Serological tests may also not show changes during early stages of disease.

5. Lack of Integrated Data Analysis:

- ✧ Current systems often analyze data in isolation (e.g., imaging or lab results), failing to capture the comprehensive picture of disease progression.
- ✧ There is a lack of systems that automatically combine image data with patient record data.

6. Limited Automation and Efficiency:

- ✧ Many diagnostic processes still rely heavily on manual interpretation, limiting efficiency and scalability.
- ✧ Many systems require large amounts of human interaction.

7. Data Heterogeneity and Variability:

- ✧ Medical imaging data can vary significantly due to differences in imaging protocols, equipment, and patient characteristics.
- ✧ Patient record data also has high variability.

8. Computational Demands:

- ✧ Deep learning models, while powerful, require significant computational resources for training and inference.

9. Interoperability Challenges:

- ✧ Deep learning models, especially complex architectures, can be "black boxes," making it difficult to understand their decision-making process.
- ✧ This lack of interpretability can hinder clinical adoption.

1.3 PROPOSED SYSTEM

This project proposes a hybrid deep learning system that integrates:

- **Convolutional Neural Networks (CNN's):** To extract spatial features from high-resolution liver biopsy images, capturing detailed pathological alterations.
- **Long Short-Term Memory (LSTM) Networks:** To analyze sequential clinical data, such as laboratory test results and patient medical history, capturing temporal dependencies and disease progression.
- **Data Fusion:** Techniques to integrate and synchronize spatial and temporal data for comprehensive analysis.
- **Automated Diagnostic Output:** A system that provides rapid, automated, and highly accurate diagnostic output, including disease classification and severity assessment.
- **Evaluation and Validation:** Rigorous evaluation of the hybrid model using performance metrics like accuracy, precision, recall, F1 score, and AUC, and comparison with traditional diagnostic methods and standalone deep learning models.
- **Deep learning techniques for image analysis:** Utilizing image segmentation, localization, detection, and classification techniques to analyze liver images from CT, MRI, and US.

Advantages of Proposed System :

- **Enhanced Accuracy:** The hybrid model, by combining spatial features from CNN's and temporal features from LSTMs, is expected to achieve higher diagnostic accuracy (as indicated by the reported 99% accuracy in the abstract) compared to traditional methods and standalone deep learning models.
- **Improved Objectivity and Reduced Inter-observer Variability:** Automation through deep learning minimizes the subjectivity inherent in manual interpretation of biopsies and imaging, leading to more consistent and reliable diagnoses.
- **Faster Diagnosis:** The automated nature of the system promises to provide rapid diagnostic outputs, reducing the time lag associated with traditional, labor-intensive methods like histopathological examination.

- **Potential for Early-Stage Detection:** By analyzing detailed image features and temporal trends in clinical data, the hybrid model may be more sensitive to subtle, early-stage pathological changes that might be missed by conventional imaging or serological tests.
- **Comprehensive Data Integration:**The system effectively integrates and synchronizes spatial information from medical images with temporal information from patient records, providing a more holistic view of the disease and its progression.
- **Automated and Efficient Diagnosis:** The proposed system aims to provide a fully or semi-automated diagnostic tool, increasing efficiency and scalability of liver disease diagnosis, particularly beneficial in resource-limited settings.
- **Leveraging Advanced Deep Learning Techniques for Image Analysis:** The system utilizes sophisticated image analysis techniques within the CNN framework, including segmentation, localization, detection, and classification, to extract maximum information from liver images.
- **Potential for Improved Prognosis:** By analyzing temporal data, the LSTM component can potentially contribute to better understanding and prediction of disease progression, aiding in prognosis.
- **Data-Driven Clinical Decision-Making:** The highly accurate and comprehensive diagnostic output can empower clinicians to make more informed and timely treatment decisions, potentially leading to improved patient outcomes.
- **Rigorous Evaluation and Validation:** The project emphasizes thorough evaluation using standard performance metrics and comparison with existing methods, ensuring the reliability and effectiveness of the proposed system.

1.4 ORGANIZATION OF THESIS

Thesis Title : Detection Of Hepatitis Virus Infection On Liver Biopsy Image Using Deep Learning

Chapter 1: Introduction

In this chapter, problem statement, existing system, proposed system and organization thesis.

Chapter 2: Literature Survey

In this chapter, the previous systems are noted and discussed about basic concepts and project related work.

Chapter 3: Feasibility Study

In this chapter, We discussed about different feasibility parts of the system.

Chapter 4: Software Requirements Specifications

In this chapter, Functional requirements, non-functional requirements, hardware and software requirements.

Chapter 5: System Modelling

This chapter gone through different UML diagrams they gives the process, design, scenarios of the system.

Chapter 6: System Implementation

The tools used, environment which has been used for development and platform, source code.

Chapter 7: Testing

Different testing strategies have been discussed.

Chapter 8: Results

This chapter contains the outputs of the system and reports generated during development and user manual.

Chapter 9: Limitations

Proposed system limitations have been discussed.

Chapter 10: Conclusion and Future Work

Conclusion of the system and future development and some suggestions are discussed.

Chapter 11: References

Gives the list of References for the system development.

CHAPTER 2

LITERATURE REVIEW

S.No	Title	Author(s)	Year	Algorithm/Focus	Disadvantages
1.	Global Burden of Liver Diseases and the Potential of AI-Driven Solutions	Asrani et al.	2019	AI-driven diagnostics	Limited healthcare infrastructure in low-income regions, challenges in widespread AI implementation
2.	Deep Learning with Convolutional Neural Networks for Enhanced NAFLD Analysis	Park et al.	2023	CNN-based Deep Learning	Needs large annotated datasets, potential biases in clinical data, interpretability challenges
3.	Revolutionizing Liver Image Analysis with Deep Learning for Malignant Lesion Detection	Survarachakan et al.	2022	U-Net, ResNet, DenseNet architectures	Dependence on high-quality annotated data, risk of model bias, need for multimodal data integration
4.	Machine Learning Approaches for Liver Disease Diagnosis: Pattern Extraction and Ethical Considerations	Khan et al.	2022	Decision Trees, SVM, Random Forests, DNNs	Ethical issues (privacy, bias), transparency and explainability challenges
5.	Advanced Segmentation Techniques for Automated Liver	Al-Kababji et al.	2023	U-Net, Mask R-CNN, Hybrid Approaches	Variability in liver anatomy, challenges in contrast differences, continuous model

S.No	Title	Author(s)	Year	Algorithm/Focus	Disadvantages
	Tissue Delineation in Medical Imaging				refinement needed
6.	Artificial Intelligence Across the Spectrum of Liver Disease Management: From Diagnosis to Treatment	Ahn et al.	2020	Decision Support Systems, Reinforcement Learning, Predictive Analytics	Regulatory hurdles, clinical adoption barriers, physician acceptance, need for explainable models

Table-1 Literature Review

2.1 BASIC CONCEPTS :

Detecting and diagnosing hepatitis virus infection on liver biopsy images using a hybrid CNN+LSTM model involves leveraging the strengths of convolutional neural networks for analyzing the visual characteristics of the biopsy images and potentially incorporating sequential patient data using long short-term memory networks for a more comprehensive diagnosis. Here are the basic concepts involved:

1. Liver Biopsy Images:

- **Definition:** A liver biopsy is a medical procedure where a small sample of liver tissue is extracted for examination under a microscope. These images provide crucial information about the liver's cellular structure and can reveal signs of inflammation, fibrosis, and viral infection.
- **Importance:** For detecting hepatitis, the biopsy images can show specific histological features indicative of viral infection, such as inflammatory cell infiltration, hepatocyte damage (ballooning degeneration, apoptosis), and the presence of viral components (though this might require specialized staining).
- **Data Source:** These images will form the primary input for the CNN part of your model. A dataset of annotated liver biopsy images, where each image is labeled as either showing signs of hepatitis virus infection or not, is crucial for training.

2. Convolutional Neural Networks (CNN's) for Image Analysis:

- Definition: CNN's are a class of deep learning models specifically designed for processing grid-like data, such as images. They consist of layers that learn hierarchical representations of visual features.
- **Key Components:**
  - ✧ Convolutional Layers: These layers use filters (small matrices) to scan the input image, detecting patterns like edges, textures, and more complex shapes. Multiple filters are used to capture diverse features.
  - ✧ **Activation Functions:** Non-linear functions (e.g., ReLU) applied after each convolutional layer to introduce non-linearity, allowing the network to learn complex relationships.
  - ✧ Pooling Layers: These layers down sample the feature maps, reducing their dimensionality while retaining important information, making the model more robust to variations in position and scale.

- ✧ Fully Connected Layers: In the final stages of the CNN, these layers take the high-level features learned by the convolutional and pooling layers and use them to make a classification decision (e.g., infected or not infected).

3. Long Short-Term Memory Networks (LSTMs) for Sequential Data (Potentially):

- Definition: LSTMs are a type of recurrent neural network (RNN) capable of learning and remembering information over long sequences of data. They are particularly useful for tasks involving time series or sequential dependencies.
- Key Components: LSTM units have a memory cell and three gates (input, output, and forget) that regulate the flow of information, allowing them to selectively remember or forget information over time.
- **Potential Application to Liver Disease Diagnosis:** While your abstract focuses on image analysis, the mention of LSTM suggests you might also be considering incorporating sequential patient data. This could include:

Time-series of liver function tests: Changes in enzyme levels (ALT, AST), bilirubin, etc., over time can be indicative of disease progression or response to treatment.

- ✧ Patient history: Previous diagnoses, medications, and other relevant medical information presented in a sequential manner.

- **Integration with CNN:** If you have access to such sequential data, the features extracted from the biopsy image by the CNN could be combined with the temporal features learned by the LSTM to make a more informed diagnosis or prognosis. The CNN output could be fed as the initial input to the LSTM, or the features from both networks could be fused at a later stage.

4. Hybrid CNN+LSTM Model:

- **Architecture:** This involves combining a CNN architecture (for image feature extraction) with an LSTM architecture (for processing sequential data, if available). The specific way these two networks are connected depends on the nature of your data.
- **Rationale:** The hybrid approach aims to leverage the strengths of both models: CNN's excel at spatial feature extraction from images, while LSTMs are powerful for capturing temporal dependencies in sequential data. This combination can potentially lead to a more accurate and comprehensive understanding of liver disease.

5. Training Data and Annotation:

- **Labeled Dataset:** You will need a dataset of liver biopsy images where each image is accurately labeled as either "hepatitis virus infection present" or "hepatitis virus infection absent." For more detailed diagnosis, you might have multiple categories for different stages or types of hepatitis.
- Data Augmentation: Techniques like rotation, flipping, and zooming can be applied to the training images to increase the size and diversity of the dataset, improving the model's generalization ability.

6. Model Training:

- **Loss Function:** A function that quantifies the difference between the model's predictions and the true labels in the training data. The goal of training is to minimize this loss. Common loss functions for classification include cross-entropy loss.
- **Optimization Algorithm:** An algorithm (e.g., Adam, SGD) used to update the model's weights during training to minimize the loss function.
- **Epochs and Batch Size:** Training involves iterating over the dataset multiple times (epochs), and in each iteration, the data is processed in smaller batches.

7. Evaluation Metrics:

- **Accuracy:** The percentage of correctly classified images.
- **Precision:** Out of all the images predicted as having hepatitis, what proportion actually had hepatitis? (True Positives / (True Positives + False Positives))
- **Recall (Sensitivity):** Out of all the images that actually had hepatitis, what proportion were correctly identified by the model? (True Positives / (True Positives + False Negatives))
- F1-Score: The harmonic mean of precision and recall, providing a balanced measure of the model's performance.
- Area Under the Receiver Operating Characteristic Curve (AUC-ROC): A measure of the model's ability to distinguish between the two classes (infected vs. not infected). An AUC of 0.99 indicates excellent performance.

2.2 PROJECT RELATED WORK :

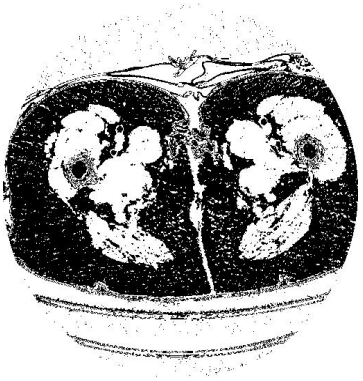


Fig: 2.2.1 Sample image from Dataset

Algorithms Used in the Module:

An algorithm is a step-by-step procedure or set of rules for solving a specific problem or accomplishing a particular task. In essence, it's a sequence of well-defined instructions that can be executed to achieve a desired outcome. Algorithms are fundamental to computer science and are used extensively in various fields, including mathematics,engineering, data science, and artificial intelligence.

1. CNN(Convolutional Neural Networks)

2. Long Short-Term Memory Networks (LSTMs)

3. Image Preprocessing Algorithms

4. Post-processing Algorithms

Step 1: Start

Step 2: Open interface

Step 3: Import libraries

Step 4: Set path for dataset

Step 5: load data set Step

Step 6: Training and testing Step

Step 7: Model performance on test data Step

Step 8: Result

CHAPTER 3

FEASIBILITY STUDY

A feasibility study is a systematic analysis and evaluation of a proposed project or business venture to assess its viability and potential for success. The purpose of a feasibility study is to determine whether the project is feasible, both technically and economically, and to identify any potential risks or challenges that may need to be addressed.

Types of Feasibility Study

A plausibility analysis evaluates the project's feasibility, therefore investors and banks should trust its objectivity. Three key types of feasibility studies analyze different subjects, as recorded below.

3.1 TECHNICAL FEASIBILITY

This assessment evaluates the organization's technical resources. It determines whether the technical team can transform ideas into functional systems and if the technical capacity matches the proposed project.Technical feasibility involves assessing the system's hardware, software, and other technical requirements.

3.2 OPERATIONAL FEASIBILITY

This evaluation involves studying whether and how the project can address the organization's needs.Operational feasibility studies also assess how a project plan meets system development's requirements analysis.

3.3 ECONOMIC FEASIBILITY

This evaluation generally includes a cost-benefit analysis to help organizations assess project viability, costs, and benefits prior to allocating funds. It also assists decision-makers with evaluating the project's financial advantages to the organization as an objective project review, further improving venture credibility.

CHAPTER 4

SOFTWARE REQUIREMENT SPECIFICATIONS

Software requirements refer to the descriptions of the functions, capabilities, and constraints of a software system that are necessary for its development, implementation, and maintenance. These requirements serve as the foundation for designing, building, and testing the software, ensuring that it meets the needs and expectations of its users and stakeholders. Software requirements can be categorized into various types, including functional requirements, non-functional requirements, and constraints.

- Feasibility Study
- Requirements Gathering
- Software Requirements Specification
- Software Requirements Validation

The basic requirements of our project are:

- Python installation
- Python IDE
- Research Papers
- Datasets
- Accuracy calculation

4.1 FUNCTIONAL REQUIREMENTS :

Functional requirements outline the specific functionalities and features that the hepatitis virus infection detection system using CNN's should possess to meet the needs of its users .

1. Image Input:

- The system shall accept medical images of liver biopsies in standard formats such as PNG, JPG or JPEG.
- The system shall support the uploading of images from local storage.

2. Hepatitis Detection:

- The system shall employ deep learning algorithms to analyze liver biopsy images and detect the presence or absence of hepatitis virus infection.
- The system shall provide a binary classification output: "Hepatitis Detected" or "No Hepatitis Detected".
- The system may be capable of identifying different stages or types of hepatitis infection and provide corresponding classifications.

3. User Interaction:

- The system shall provide a user-friendly interface for users to upload and process liver biopsy images.
- Users shall be able to view the uploaded image.
- The system shall accept medical images of liver biopsies in standard formats such as PNG, JPG or JPEG.
- The system may display a confidence score or probability associated with the prediction.

4. Reporting:

- The system shall generate a report summarizing the analysis of the uploaded image.
- Reports shall include the uploaded image filename and the classification result.
- Reports may include a confidence score or probability of the prediction.
- Reports may include patient information and timestamps.
- Reports shall be exportable in a common format (e.g., PDF).

5. Integration:

- The system may integrate with existing medical imaging systems (e.g., PACS) for seamless image retrieval.The system may support interoperability with other healthcare IT systems through standard interfaces (e.g., HL7) for data exchange.

4.2 NON-FUNCTIONAL REQUIREMENTS:

In systems engineering, a non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system, rather than specific behaviors.They are contrasted with functional requirements that define specific behavior or functions.The non-functional requirements can be considered as quality attributes of a system.

1. Performance:

- The system shall process an uploaded liver biopsy image and provide a detection result within a maximum response time of 10 seconds.

2. Accuracy:

- The deep learning model shall achieve a minimum accuracy of 95% in detecting hepatitis virus infection on a held-out test dataset, compared to expert annotations.
- The system shall aim to minimize false positives (incorrectly identifying infection) and false negatives (failing to identify infection).

3. Reliability:

- The system shall be available for use at least 99% of the time during intended operational hours.
- The system should include basic error handling to prevent crashes during image processing.

4. Security:

- Access to the system shall be restricted to authorized users (if deployed in a multi-user environment).
- If patient data is involved, the system shall comply with relevant healthcare data security standards.

5. Usability:

- The system shall provide a clear and intuitive user interface for medical professionals with basic computer skills.

6. Scalability:

- The system architecture should be designed to accommodate a growing number of users and images if wider deployment is planned.

7. Regulatory Compliance:

- The system development should consider relevant regulatory requirements for medical devices and software in the intended jurisdiction.

4.2.1 PERFORMANCE REQUIREMENTS:

Performance requirements specify the criteria that the hepatitis virus infection detection system using CNN must meet in terms of speed and accuracy.

- Speed: The CNN model should process a single liver biopsy image and provide a prediction within a reasonable time frame (as specified in Non-Functional Requirements - Performance).
- Accuracy: The model's predictions must be highly accurate in identifying the presence or absence of hepatitis virus infection to be clinically useful (as specified in Non-Functional Requirements - Accuracy).
- Reliability: The model should provide consistent and dependable results across different biopsy images.

4.2.2 HARDWARE REQUIREMENTS:

- Computer: A standard computer system capable of running the necessary software. Specific requirements (CPU, RAM, GPU) may vary depending on the complexity of the model and the intended deployment environment (local workstation vs. server).
- Storage: Adequate storage capacity to store the project code, datasets, and trained models.
- (Recommended for Training) GPU Acceleration: A dedicated NVIDIA GPU with sufficient memory can significantly speed up the training process of the deep learning model.
- CPU: Intel Core i5 or equivalent (minimum for development and potential deployment).
- RAM: 8 GB or 16 GB recommended for development and efficient processing.

4.2.3 SOFTWARE REQUIREMENTS:

The purpose of System Requirements Analysis is to obtain a thorough and detailed understanding of the business need and break it down into discrete requirements. During System Requirements Analysis, the framework for the application is developed, providing the foundation for all future design and development efforts.

1. **Operating System:** Windows, macOS, or Linux (compatible with the chosen development environment).
2. **Text Editor or Integrated Development Environment (IDE):** For writing and editing code (e.g., Visual Studio Code, PyCharm, Jupyter Notebook).
3. **Web Browser:** A modern web browser for interacting with the user interface (if a web-based application).
4. **Python:** A programming language used for developing the deep learning model and any associated application logic.
5. **Libraries and Dependencies:** Install the necessary Python libraries:
   - TensorFlow or PyTorch: Deep learning framework for building and training the CNN model.
   - Keras: High-level API for TensorFlow (if using TensorFlow).
   - NumPy: For numerical computations.
   - Open CV (cv2): For image processing tasks.
   - sci kit-learn: For evaluation metrics and data splitting.
   - Matplotlib and/or Seaborn: For data visualization and plotting results.
   - Flask or Django (if developing a web-based user interface).
   - Gradio (for creating a simple interactive demo interface - as seen in your code).

4.2.3.1 INTRODUCTION TO PYTHON:

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. It was created by Guido van Rossum and first released in 1991. Python emphasizes code readability and productivity, making it popular among developers for various applications, including web development, data analysis, artificial intelligence, scientific computing, automation, and more.

Here's a brief introduction to some key aspects of Python:

1. **Simple and Readable Syntax:** Python's syntax is designed to be easy to read and write, resembling pseudo-code. It uses indentation to define code blocks, eliminating the need for braces or semicolons. This simplicity makes Python particularly suitable for beginners and promotes code consistency and maintainability.
2. **Interpreted and Interactive:** Python is an interpreted language, meaning that code is executed line by line by the Python interpreter. This allows for rapid development and testing, as code changes can be immediately executed without the need for compilation. Python also supports interactive mode, where code can be executed interactively in a REPL (Read-Eval-Print Loop) environment.
3. **Dynamic Typing and Strong Typing:** Python is dynamically typed, meaning that variable types are determined at runtime based on the assigned values. It also enforces strong typing, meaning that variables cannot change their type implicitly, promoting code safety and clarity.
4. **Rich Standard Library:** Python comes with a comprehensive standard library that provides a wide range of modules and packages for various tasks, such as file I/O, networking, data manipulation, mathematics, and more. This extensive library reduces the need for external dependencies and simplifies development.
5. **Object-Oriented Programming (OOP):** Python supports object-oriented programming paradigms, allowing developers to define classes and create objects with attributes and methods. OOP concepts such as inheritance, encapsulation, and polymorphism are supported, enabling the development of modular and reusable code.
6. **Functional Programming Features:** Python also supports functional programming features, such as higher-order functions, lambda functions, and list comprehensions. These features enable concise and expressive code for tasks involving transformations and iterations.

CHAPTER 5

SYSTEM MODELING

5.1REQUIREMNETS MODELING

- System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling may represent a system using graphical notation, e.g. the Unified Modeling Language (UML).
- System modeling helps the analyst to understand the functionality of the system and models are used to communicate with customers.

5.1.1 CLASS DIAGRAM :

A class diagram is a type of static structure diagram in the Unified Modeling Language (UML) that represents the structure and relationships of classes with in a system.

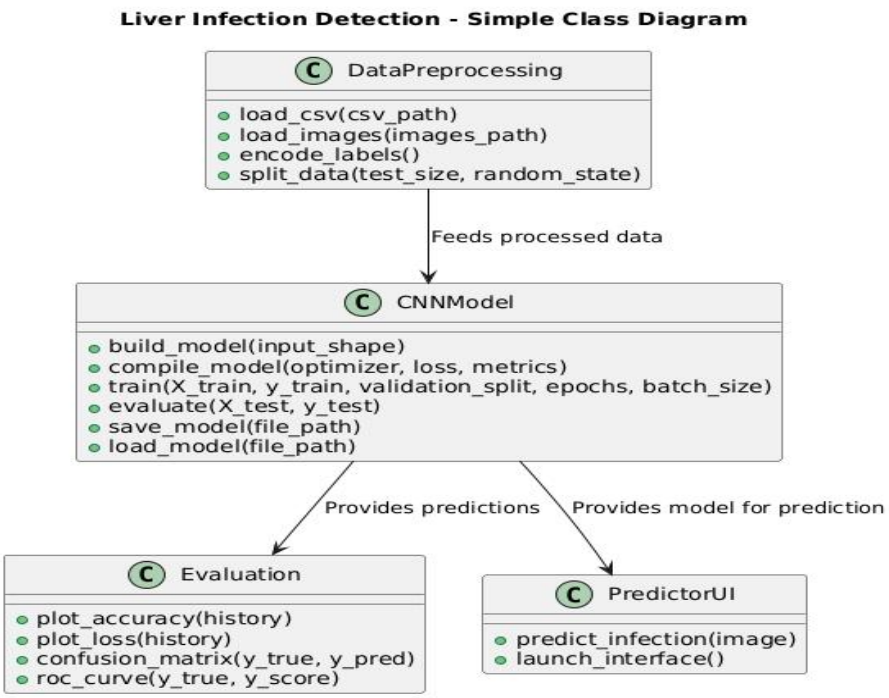


Fig 5.1.1 Class Diagram

Class Relationships :

- Model is fed preprocessed data from DataPreprocessing.
- Model outputs predictions for evaluation to Evaluation.
- Evaluation uses the output predictions from Model to generate various plots and metrics for assessment.

5.1.2 USE CASE DIAGRAMS :

A use case diagram is a type of behavioural diagram in the Unified Modelling Language (UML) that represents the interactions between actors (users or external systems) and the system under consideration.

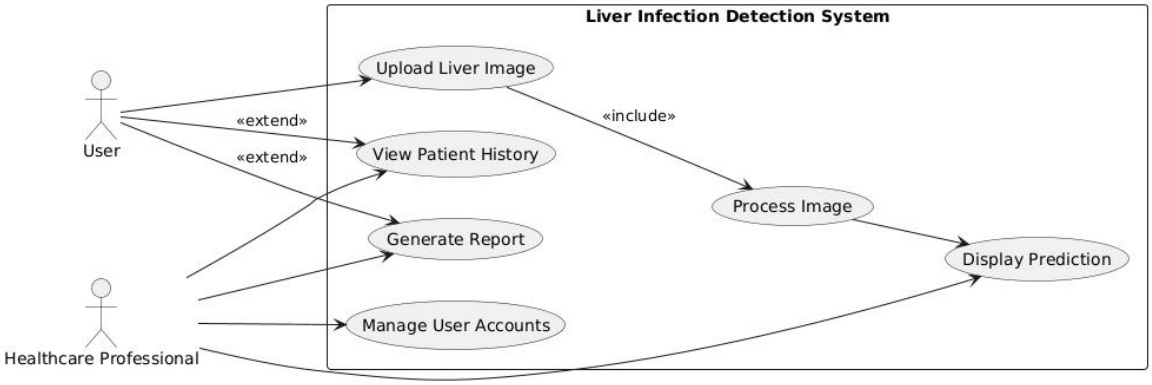


Fig 5.1.2 Use Case Diagram

For a General User:

1. **Upload Liver Image:** The user has the option to upload a liver image to the system.
2. **Process Image:** This action is included whenever a liver image is uploaded. The system will analyze the image to detect potential signs of infection.
3. **Display Prediction:** Following the image processing, the system will display a prediction
4. **View Patient History:** The user can optionally extend the basic functionality to view the patient's history. This suggests that the system might store past scans and predictions.
5. **Generate Report:** The user can also optionally extend the basic functionality to generate a report, which might summarize the findings and history.

For a Healthcare Professional:

1. **Manage User Accounts:** Healthcare professionals have the capability to manage user accounts within the system. This likely includes creating, deleting, and modifying user profiles.
2. They also have access to the **Generate Report** functionality, similar to a general user.

5.1.3 SEQUENCE DIAGRAM :

A sequence diagram is a type of behavioural diagram in the Unified Modelling Language (UML) that represents the interactions between objects or components within a system over time. It illustrates the sequence of messages exchanged between objects to accomplish a specific behaviour or scenario.

Lifelines: Represent objects participating in the sequence, shown as vertical lines. **Messages:** Arrows between lifelines indicate messages exchanged between objects. **Activation Bars:** Horizontal lines on lifelines denote the duration of object activity during message processing.

Synchronous Messages: Represented by solid arrows, synchronous messages imply direct and immediate communication between objects.

Asynchronous Messages: Indicated by dashed arrows, asynchronous messages signify delayed or non-blocking communication.

Return Messages: Arrows back to the sender represent return messages, conveying responses or results.

Self-References: Objects can send messages to themselves, depicted by loops or self-referencing arrows.

Interaction Occurrences: Rectangular frames denote nested sequences of interactions within the main sequence diagram.

Time Progression: Time progresses from top to bottom, with messages flowing sequentially over the lifelines.

Analysis and Design: Sequence diagrams aid in system analysis, design, and understanding of object interactions, facilitating communication among stakeholders.

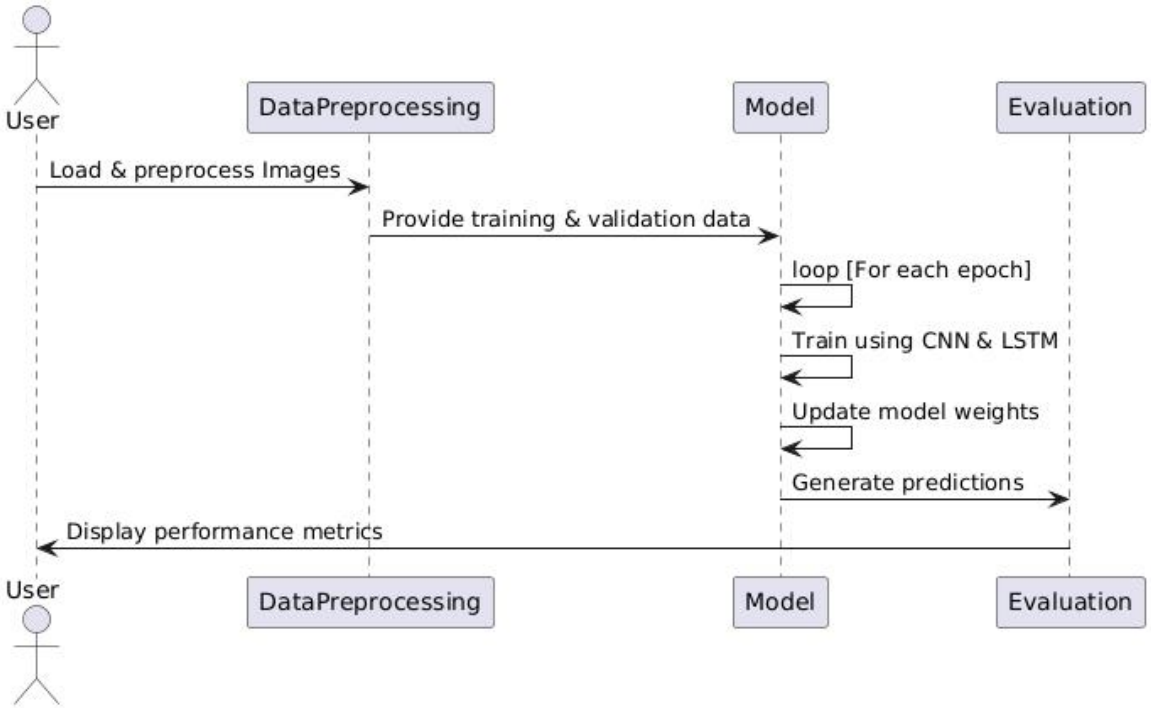


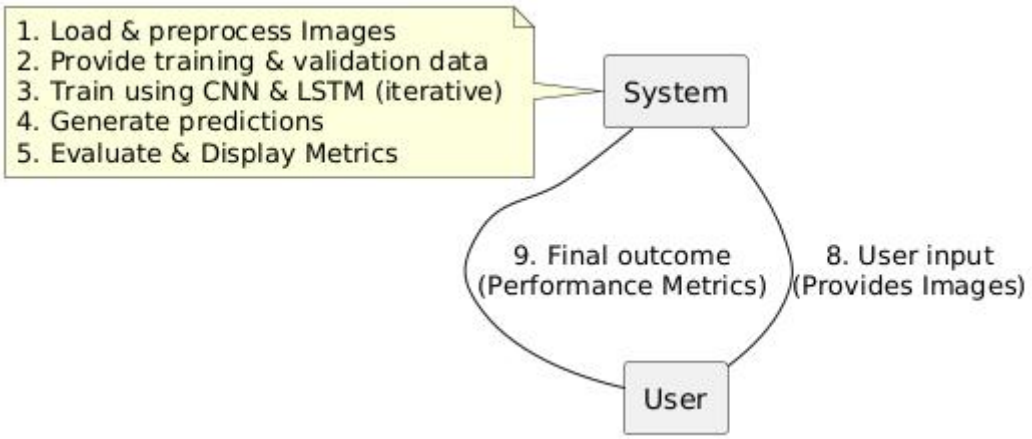
Fig 5.1.3 Sequence Diagram

Step-by-Step Workflow

1. User provides data → DataPreprocessing loads and preprocesses images/masks.
2. DataPreprocessing provides prepared data → Model receives training and validation sets.
3. Model trains iteratively → CNN & LSTM layers learn and update model weights per epoch.
4. Trained Model predicts → Generates outputs on the validation data.
5. Predictions are evaluated → Evaluation calculates and sends performance metrics to the User.

5.1.4 COLLABORATION DIAGRAM :

A collaboration diagram, in the Unified Modeling Language (UML), is a type of interaction diagram that shows the message flow between the objects or roles that participate in a specific collaboration to achieve a particular goal.



5.1.4 Collaboration Diagram

Major Component Interactions

- The diagram shows a workflow between a "System" and a "User" for image-related machine learning.
- The "System" first loads and preprocesses input images.
- It then uses training and validation data to learn patterns.
- The learning process involves iterative training with CNN and LSTM networks.
- After training, the system generates predictions on new data.
- Finally, the system evaluates its performance and displays relevant metrics.
- The "User" provides images as input to the system.
- The "System" returns the final performance metrics as the outcome to the user.
- The process is cyclical, suggesting potential for repetition and refinement.

5.1.4 ACTIVITY DIAGRAM :

Activity diagram is essentially a fancy flowchart: Activity and state diagrams are related. State chart diagram focuses on s undergoing a process. An activity diagram focuses on the flow of activities involved in a single process. The activity diagram shows the activities depend on one another.

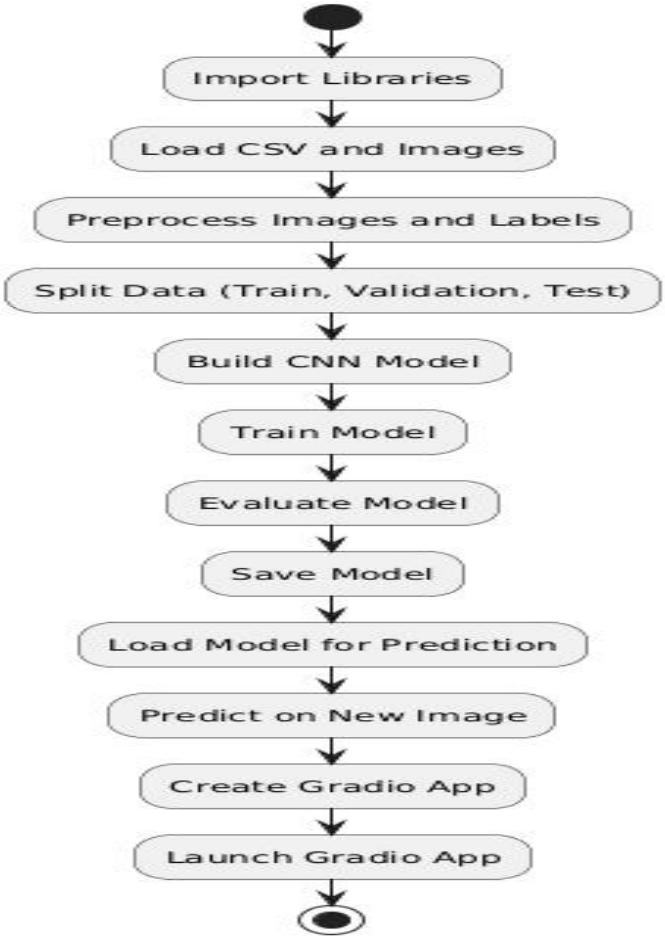


Fig 5.1.4 Activity Diagram

The Activity diagram outlines the steps to build and deploy an image classification model. It starts by importing libraries and loading/preprocessing data (images and labels), then splits the data for training, validation, and testing. A CNN model is built, trained, and evaluated. The trained model is saved and later loaded to predict the class of new images. Finally, a Gradio app is created and launched to provide a user interface for interacting with the model.

5.1.5 DATA FLOW DIAGRAM :

A Data Flow Diagram (DFD) is a graphical representation of the flow of data within a system, illustrating how data moves between processes, data stores, and external entities. It provides a visual depiction of the processes involved in a system and the interactions between them, focusing on the movement and transformation of data rather than the control flow or sequence of operations.

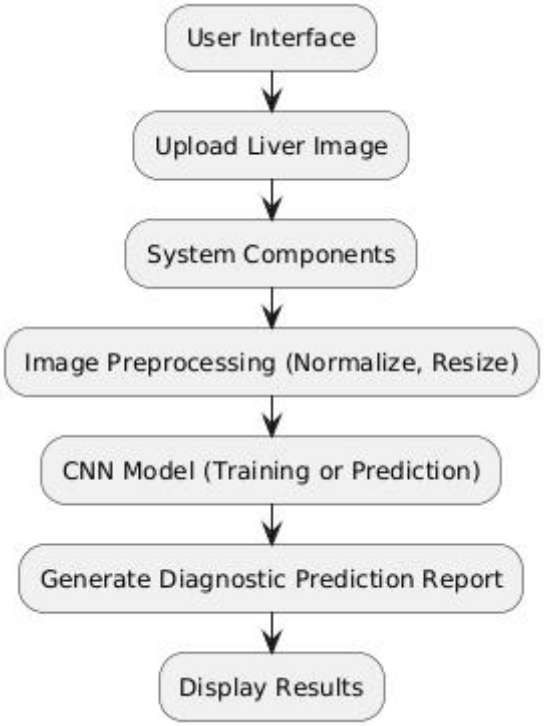


Fig 5.1.5 Data Flow Diagram

5.1.6 DEPLOYMENT DIAGRAM :

A deployment diagram in the context of machine learning illustrates how a trained model or machine learning system is deployed and interacts within a production environment. It outlines the physical or virtual components where the model resides and how they communicate with each other and external systems. Typically, it depicts servers, databases, APIs, and other infrastructure elements, along with the deployment configurations required for effective operation.

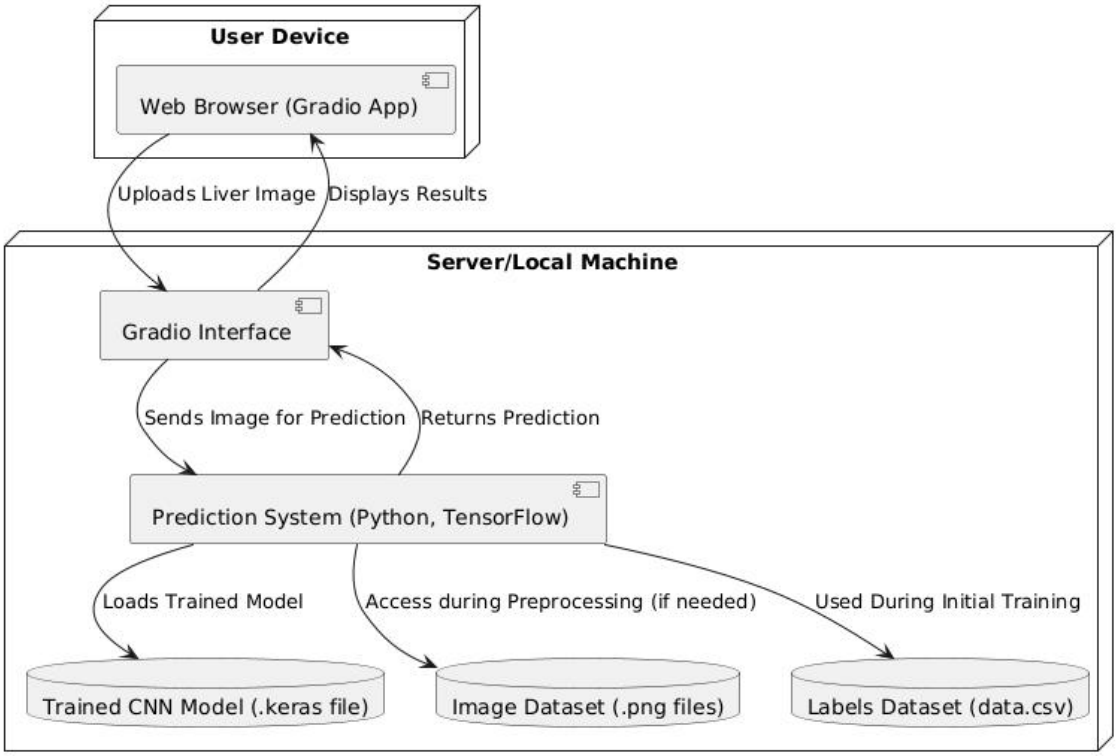


Fig 5.1.6 Deployment Diagram

5.1.7 MODULES DESCRIPTION

In the proposed deep learning-based model for detecting hepatitis virus infection, the system architecture is composed of several specialized modules. Each module handles a specific task in the processing pipeline from loading biopsy images to outputting a prediction score that indicates infection presence.

Liver biopsy image datasets (along with their corresponding masks) are used as input to the model. These images are preprocessed, annotated with binary labels (Infection/No Infection), and split into training and validation sets for model development.

Module 1: Environment Setup & Dependency Installation

- **Function**: Install all necessary Python libraries like TensorFlow, OpenCV, Matplotlib, and check GPU availability.
- **Purpose**: Ensures the environment is ready for development and model training.

Module 2: Dataset Preparation & Preprocessing

- **Function**: Load liver biopsy images and their corresponding masks. Resize, normalize, and convert them to NumPy arrays.
- **Purpose**: Clean and format data for model ingestion.

Module 3: Train-Validation Split

- **Function**: Split the preprocessed dataset into training (90%) and validation (10%) sets. Assign binary labels for infected and non-infected samples.
- **Purpose**: Create datasets for training and evaluation.

Module 4: Model Architecture & Compilation

- **Function**: Define the CNN using layers such as Conv2D, MaxPooling2D, Flatten, and Dense. Compile with Adam optimizer and binary_crossentropy loss function.
- **Purpose**: Create a trainable architecture for binary classification.

Module 5: Model Training & Early Stopping

- **Function**: Train the model using batch size of 16 for 20 epochs. Incorporate early stopping to prevent overfitting.
- **Purpose**: Optimize the model's performance on training data while generalizing well on unseen data.

Module 6: Model Evaluation & Performance Metrics

- **Function**: Evaluate the model using accuracy, confusion matrix, and AUC-ROC score. Visualize model performance.
- **Purpose**: Analyze how well the model performs and identify areas for improvement.

Module 7: Sample Image & Mask Display

- **Function**: Load and visualize a few sample biopsy images along with their segmentation masks.
- **Purpose**: Validate preprocessing visually and ensure correct image-mask alignment.

Module 8: Model Saving & Deployment

- **Function**: Save the trained model in `.h5` format. Prepare the model for real-time inference.
- **Purpose**: Make the model reusable and deployable.

Module 9: Gradio Interface for Inference

- **Function**: Create a simple Gradio web UI that allows users to upload biopsy images, runs the model on them, and returns a probability score for infection.
- **Purpose**: Enable easy access and interaction with the model for end-users without coding knowledge.

5.2 DESIGN MODELING:

Design modelling creates visual blueprints of the Hepatitis Virus Detection software, aiding analysis, visualization, and communication of its structure and behaviour for medical professionals. It employs structural, behavioural, and architectural techniques to depict static elements, dynamic interactions, and high-level structure. Data modelling handles medical image data and predictions, while UI modelling focuses on an intuitive interface.

Working of Design Modelling in Software Engineering:

Requirement Analysis: Analyze medical and technical requirements (image upload, prediction using CNN/LSTM via TensorFlow/Keras, displaying results and metrics like accuracy, loss, confusion matrix, ROC curve; usability, performance, security).

Identifying Design Goals: Establish objectives like modularity (UI, preprocessing, model, evaluation), usability (for medical experts), efficiency (fast prediction), maintainability (for model updates), and performance (efficient Deep Learning inference).

Selecting Modelling Techniques: Choose appropriate models:

- **Structural:** Component Diagrams (high-level components), Class Diagrams (class details).
- **Behavioural:** Activity Diagrams (workflows), Sequence Diagrams (component interactions).
- **Architectural:** Layered Architecture, Deployment Diagrams.
- **Data:** Conceptual Data Model (medical image data, predictions).
- **UI:** Wireframes (interface design).

Creating Design Models: Develop visual representations of the system's architecture, components (UI with Gradio, Preprocessing, TensorFlow/Keras Model, Prediction Engine, Evaluation), interactions (image upload triggering processing and prediction), and data flow (medical images to predictions and metrics).

Iterative Refinement: Refine models based on feedback from medical professionals, prototyping results, and evolving understanding of medical image features and Deep Learning model performance.

Verification and Validation: Ensure models meet requirements through reviews with medical experts and technical inspections, checking for consistency, completeness, and correctness.

Documentation and Communication: Use models to document design decisions (e.g., CNN architecture choice), rationale (e.g., preprocessing steps), and trade-offs (e.g., accuracy vs. speed) for developers (implementation with TensorFlow/Keras and Gradio), testers (test case design), project managers, and medical customers (understanding system use).

5.2.1 SYSTEM ARCHITECTURE:

This system architecture outlines the process of detecting Hepatitis indicators from medical images (likely liver scans) using a Convolutional Neural Network (CNN). Here's an overview:

1. **Input Data:** The system takes two primary inputs:
   - **Input Images:** A dataset of medical images (e.g., X-rays, CT scans, ultrasound) of the liver.
2. **Data Processing:** Before feeding into the CNN, the input data undergoes preprocessing:
   - **Image Preprocessing:** Operations are performed on the input images to standardize them and enhance relevant features. This might include resizing, normalization (as explicitly shown), noise reduction, or other domain-specific techniques.
   - **Train-Test Split:** The combined dataset of normalized images and normalized masks is then divided into training and testing (or validation) sets. The training set is used to teach the CNN, while the testing set evaluates its performance on unseen data.
3. **Convolutional Neural Network (CNN) Architecture:** The core of the detection system is a deep convolutional neural network designed to learn hierarchical features from the medical images:
   - **Convolutional Layers (Conv2D Layer 1, Conv2D Layer 2, Conv2D Layer 3, Conv2D Layer 4):** These layers are the fundamental building blocks of the CNN. They apply learnable filters to the input feature maps, detecting spatial patterns such as edges, textures, and subtle abnormalities that might indicate Hepatitis.

     The architecture shows multiple convolutional layers stacked, allowing the network to learn increasingly complex features.

- **Max Pooling Layers (MaxPooling2D):** Following each convolutional layer (except potentially the last), max-pooling layers are used to down-sample the feature maps. This reduces the spatial dimensions, making the network more robust to variations in the position and scale of features and reducing computational complexity.
- **Flatten Layer:** After the series of convolutional and pooling layers, the high-dimensional feature maps are flattened into a one-dimensional vector. This prepares the learned features to be fed into the fully connected (Dense) layers.

3. Dense Layers: The flattened features are then processed by fully connected layers:

- **Dense Layer:** A standard fully connected layer that learns global relationships between the extracted features.
- **Dropout Layer:** A regularization technique applied to the dense layer to prevent overfitting. During training, a random proportion of neurons in this layer are temporarily ignored, forcing the network to learn more robust and generalizable features.
- **Output Layer:** The final fully connected layer that produces the prediction. For a binary classification task (e.g., Hepatitis detected/not detected), this layer would typically have a single neuron with a sigmoid activation function, outputting a probability between 0 and 1.

4. Training: The CNN is trained using the training data to learn the mapping between the input images and the corresponding masks (or labels derived from the masks):

- **Binary Cross-Entropy Loss:** This is a common loss function used for binary classification tasks. It measures the difference between the network's predicted probabilities and the true labels, guiding the learning process to minimize this difference.
- **Adam Optimizer:** An adaptive learning rate optimization algorithm used to update the weights of the CNN during training in order to minimize the loss function efficiently.
- **Early Stopping:** A regularization technique used to prevent overfitting. The training process is monitored on a validation set, and if the performance on this set stops improving (or starts degrading), the training is stopped early, preventing the model from memorizing the training data.

5. Evaluation: After training, the performance of the CNN is evaluated on the unseen testing data:

- **Predictions:** The trained CNN generates predictions (probabilities of Hepatitis presence) for the images in the evaluation set.
- **Performance Metrics:** Various metrics are calculated to assess the model's accuracy, precision, recall, F1-score, etc.
- **ROC-AUC (Receiver Operating Characteristic - Area Under the Curve):** A graphical plot and a metric that evaluates the model's ability to distinguish between the two classes (Hepatitis present/absent) across different classification thresholds.
- **Confusion Matrix:** A table that summarizes the performance of the classification model by showing the counts of true positives, true negatives, false positives, and false 1 negatives.

7. Output (Implicit): The final output of the system, after training and evaluation, would be a trained CNN model capable of taking a new medical image as input and predicting the likelihood of Hepatitis indicators being present. The evaluation metrics would provide an understanding of the model's expected performance on unseen data.

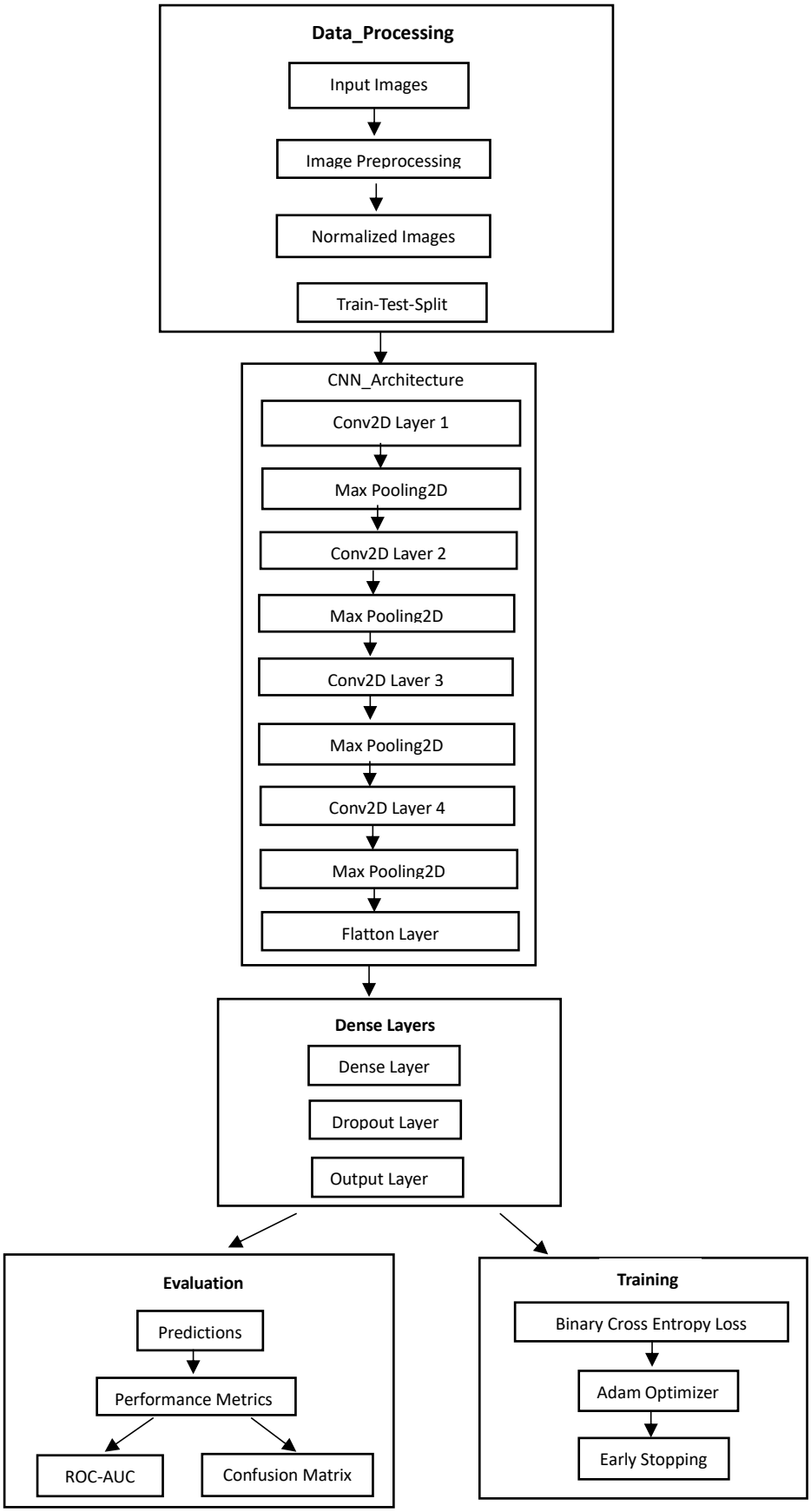


Fig 5.2.1 System Architecture

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 DEVELOPMENT ENVIRONMENT AND TOOLS:

- In the development environment for machine learning, practitioners utilize adiverse set of tools and platforms tailored to streamline the entire workflow, from data preprocessing to model deployment.
- Integrated Development Environments (IDEs) like Jupyter Notebooks, PyCharm, and Visual Studio Code provide interactive interfaces conducive to experimentation and rapid prototyping. Cloud-based platforms such as Google Colab, Amazon Sage-Maker, and Microsoft Azure Machine Learning offer scalable computing resources, pre-configured environments, and managed services for deploying machine learning workflows seamlessly.

- **Integrated Development Environments (IDEs):**

1. Google Colab: It is free Cloud based platform and offers pre-installed libraries like TensorFlow and PyTorch, allowing users to build and train deep learning models without worrying about hardware constraints. Additionally, Colab provides GPU and TPU acceleration, enabling faster model training times, making it an ideal choice for beginners and professionals alike to experiment with and deploy deep learning projects easily.

2. Visual Studio Code: Visual Studio Code (VS Code) is a lightweight yet powerful code editor loved by developers worldwide for its speed and flexibility. It offers a wide range of features, including syntax highlighting, code completion, and integrated Git support.

3. Anaconda: Anaconda Navigator is a graphical interface included in the Anaconda distribution, which allows users to manage packages, environments,and launches various tools such as Jupyter Notebook, Spyder, and VS Code. It simplifies the setup and management of Python and R environments, making it popular among data scientists and researchers for its ease of use and comprehensive package ecosystem.

Jupyter Notebook: Jupyter Notebook is an open-source web application that allows users to create and share documents containing live code, equations, visualizations, and narrative text. It supports multiple programming languages, including Python, R, and Julia, making it a versatile tool for data analysis, machine learning, and scientific computing. Jupyter Notebook printer active and exploratory coding workflows, enabling users to run code snippets in real-time, visualize data, and document their analyses in a single interactive document.Its popularity stems from its user-friendly interface, flexibility, and support for reproducible research and collaborative work.

- **TOOLS:**

1. GitHub: GitHub is a widely used platform for version control and collaboration, allowing developers to host and share code repositories seamlessly. GitHub provides a robust platform for hosting, reviewing, and contributing to software projects, facilitating seamless collaboration among developers worldwide.

2. Stream lit: Streamlit is a user-friendly framework enabling rapid development of data-driven web applications directly from Python scripts. Streamlit is a simple and efficient Python library for creating interactive web applications for machine learning, deep learning and data science projects with minimal code.

- **Libraries:**

1. TensorFlow and Keras: These are the core deep learning libraries used for building and training the Convolutional Neural Network (CNN) model. TensorFlow provides the backend for numerical computation and GPU acceleration, while Keras offers a high-level API for defining and training neural networks. The code demonstrates the use of Sequential, Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization, Activation, Adam, and EarlyStopping from Keras.

2. NumPy: Used extensively for numerical operations, especially for handling image data as multi-dimensional arrays and for array manipulations during preprocessing and model predictions.

3. CV2 (OpenCV): Employed for image processing tasks, specifically for reading images (cv2.imread) and resizing them (cv2.resize).

4. Matplotlib: Used for creating static, interactive, and animated visualizations in Python, as seen in plotting the training/validation accuracy and loss, the confusion matrix, and the ROC curve.

5. Seaborn: Built on top of Matplotlib, Seaborn provides a high-level interface for drawing attractive and informative statistical graphics, as used for visualizing the confusion matrix with a heatmap.

6. Sci-kit-learn (sklearn): Utilized for splitting the dataset into training and validation sets (train_test_split) and for evaluating the model's performance using metrics like classification_report, confusion_matrix, and roc_curve, roc_auc_score.

7. Glob: Used for finding all the path names matching a specified pattern, likely employed to easily load image and mask file paths from directories.

8. Zip file: Used for extracting the contents of the Dataset.zip file, making the image and mask data accessible for the project.

9. OS: The os module is used for interacting with the operating system, such as setting environment variables (os.environ) and joining file paths (os.path.join).

10. Gradio: As mentioned under "Tools," Gradio is the library used to build the interactive web interface for deploying the trained model.

6.2 CODING:

Coding involves translating human instructions into machine-readable language, enabling computers to perform specific tasks and solve problems efficiently. It encompasses writing, testing, debugging, and maintaining code to create software applications, websites, and other digital solutions that power our modern world.

6.2.1 SOURCE CODE:

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, confusion_matrix, roc_curve,
roc_auc_score
os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'  # Suppress TensorFlow info and
warnings
import tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.optimizers import Adam
# Check if GPU is available
print(f"TensorFlow version: {tf.__version__}")
print("GPU Available: ", "Yes" if tf.config.list_physical_devices('GPU') else "No")

#  Load Dataset CSV
csv_path = R"C:\Users\LENOVO\OneDrive\Desktop\ Heptitis virus\NEW
PROJECT\data.csv"  # Uploaded file
df = pd.read_csv(csv_path)
```

```
#  Define Image Path
images_path = R"C:\Users\LENOVO\OneDrive\Desktop\ Heptitis virus\NEW
PROJECT\images"  # Replace with your actual folder name

# Parameters
img_size = (128, 128)   # Resize all images to 128x128
num_channels = 3       # RGB Images

X = []
y = []

for index, row in df.iterrows():
    img_file = os.path.join(images_path, f"{row['ID']}.png")  # Corrected extension
    if os.path.exists(img_file):
        img = load_img(img_file, target_size=img_size)
        img_array = img_to_array(img) / 255.0
        X.append(img_array)
        y.append(row['Infection'])  # Label column
    else:
        print(f"Image {img_file} not found!")  # Shouldn't print now

X = np.array(X)
le = LabelEncoder()
y_encoded = le.fit_transform(y)
y_categorical = to_categorical(y_encoded)

# 5. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y_categorical, test_size=0.2,
random_state=42)
X_val, X_test, y_val, y_test = train_test_split(X_test, y_test, test_size=0.5,
random_state=42) # Corrected split

print(f"Training data shape: {X_train.shape}, Validation data shape: {X_val.shape}")
```

```
print(f"Test data shape: {X_test.shape}")

# 6. Build CNN Model
model = Sequential([
    tf.keras.layers.Input(shape=(128, 128, 3)),
    Conv2D(32, (3, 3), activation='relu'),# input_shape=(img_size[0], img_size[1],
num_channels)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(len(le.classes_), activation='softmax')
])

model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])

model.summary()

# 7. Train the Model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.1)

# Plot the training and validation loss/accuracy
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.legend()
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.legend()
plt.title('Training and Validation Loss')

plt.show()

# Make predictions on the validation set
y_pred_probs = model.predict(X_val)
y_pred = np.argmax(y_pred_probs, axis=1)
y_true = np.argmax(y_val, axis=1)

# Classification report and confusion matrix
print(classification_report(y_true, y_pred, target_names=le.classes_))
conf_matrix = confusion_matrix(y_true, y_pred)
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=le.classes_,
yticklabels=le.classes_)
plt.title('Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()

# AUC-ROC curve
fpr, tpr, thresholds = roc_curve(y_val[:, 1], y_pred_probs[:, 1]) # Assuming binary
classification and positive class is index 1
roc_auc = roc_auc_score(y_val[:, 1], y_pred_probs[:, 1])
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'AUC = {roc_auc:.2f}')
plt.plot([0, 1], [0, 1], linestyle='--')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc='lower right')
plt.show()
```

```
# 8. Evaluate the Model
test_loss, test_accuracy = model.evaluate(X_test, y_test)
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')

# Save Model
model.save(r 'C:\Users\LENOVO\OneDrive\Desktop\Heptitis virus\NEW
PROJECT\daliver_cnn_model.keras')

from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import numpy as np

# Load trained model
model = load_model(r"C:\Users\LENOVO\OneDrive\Desktop\Heptitis virus\NEW
PROJECT\daliver_cnn_model.keras")

# Define image path to predict (you can change this)
new_image_path = r"C:\Users\LENOVO\OneDrive\Desktop\Heptitis virus\NEW
PROJECT\images\9.png"  # Example

# Preprocess the new image
img = load_img(new_image_path, target_size=img_size)
img_array = img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)  # Shape (1, 128, 128, 3)

# Predict
prediction = model.predict(img_array)
predicted_class = np.argmax(prediction)

# Decode label
predicted_label = le.inverse_transform([predicted_class])[0]

print(f"Predicted class for 9.png: {predicted_label}")
```

```
# Define image path to predict (you can change this)
new_image_path = r"C:\Users\LENOVO\OneDrive\Desktop\Heptitis virus\NEW
PROJECT\images\240.png"  # Example

# Preprocess the new image
img = load_img(new_image_path, target_size=img_size)
img_array = img_to_array(img) / 255.0
img_array = np.expand_dims(img_array, axis=0)  # Shape (1, 128, 128, 3)

# Predict
prediction = model.predict(img_array)
predicted_class = np.argmax(prediction)

# Decode label
predicted_label = le.inverse_transform([predicted_class])[0]

print(f"Predicted class for 240.png: {predicted_label}")

import gradio as gr
from tensorflow.keras.models import load_model
from tensorflow.keras.preprocessing.image import img_to_array
import numpy as np
from PIL import Image

# === Load trained model ===
model = load_model(r"C:\Users\LENOVO\OneDrive\Desktop\Modified Heptatis
virus\NEW PROJECT\daliver_cnn_model.keras")
img_size = (128, 128)

# === Define hardcoded classes (from your LabelEncoder) ===
class_names = list(le.classes_)  # Get class names from the fitted LabelEncoder
# === Prediction function ===
```

```
def predict_infection(image):
    try:
        image = image.resize(img_size).convert('RGB')  # Resize & convert to RGB
        img_array = img_to_array(image) / 255.0
        img_array = np.expand_dims(img_array, axis=0)

        prediction = model.predict(img_array)
        predicted_index = np.argmax(prediction)
        predicted_label = class_names[predicted_index]
        confidence = round(np.max(prediction) * 100, 2)

        print("Prediction Raw Output:", prediction)
        print("Predicted Index:", predicted_index)
        print("Predicted Label:", predicted_label)

        return f"    Prediction: {predicted_label} (Confidence: {confidence}%)"

    except Exception as e:
        return f"❌ Error: {str(e)}"
# === Gradio Interface ===
interface = gr.Interface(
    fn=predict_infection,
    inputs=gr.Image(type="pil", label="Upload Liver Image (.png)"),
    outputs=gr.Textbox(label="Prediction Result"),
    title="    Liver Infection Detection using CNN",
    description="Upload a liver scan image and get back 'No Virus' or 'Virus'
classification."
)

interface.launch()
```

CHAPTER 7

TESTING

7.1 INTRODUCTION TO TESTING:

Testing is a critical phase in the software development life cycle, serving as a cornerstone for ensuring the quality, reliability, and functionality of software applications. It involves a systematic process of executing software components or systems to evaluate their behaviour and verify that they meet specified requirements. The primary goal of testing is to identify defects, errors, or deviations from expected behaviour, allowing developers to rectify issues and deliver a robust and dependable product to end-users.

At its core, testing is about validating the correctness and completeness of software solutions. This process encompasses various activities, including designing test cases, executing tests, analyzing results, and tracking defects. Test cases are meticulously crafted to cover different scenarios, inputs, and conditions to assess the software's functionality comprehensively. Through rigorous testing, developers aim to uncover issues such as bugs, performance bottlenecks, security vulnerabilities, and usability concerns, mitigating risks and ensuring a smooth user experience.

It begins in the early stages of development with unit testing, where individual components or modules are tested in isolation to ensure they function as intended. As development progresses, integration testing verifies the interactions between these components, while system testing evaluates the entire system's behaviour in a holistic manner. User acceptance testing (UAT) involves validating the software against user requirements and expectations, ensuring it meets the end-users' needs effectively.

The importance of testing extends beyond identifying defects to instilling confidence in software quality and reliability. It provides stakeholders with assurance that the software behaves predictably, performs optimally, and meets business objectives. Additionally, testing contributes to cost reduction by detecting issues early in the development process.

- Testing helps validate the robustness of software solutions under various real-world conditions, including different operating environments, user interactions, and data inputs.
- Through regression testing, software developers ensure that changes or updates to the code base do not inadvertently introduce new defects or regressions in existing functionality.
- Automated testing tools and frameworks streamline the testing process, enabling faster feedback cycles, increased test coverage, and improved overall efficiency in software development.

7.1.1 TESTING STRATEGIES:

Testing strategies refer to the comprehensive plans and methodologies devised by software development teams to guide their testing efforts throughout the software development life cycle. These strategies encompass various approaches, techniques, and best practices aimed at ensuring the quality, reliability, and effectiveness of the software being developed. Some common testing strategies are:

1. Unit Testing: Unit testing is a testing technique where individual units or components of a software application are tested in isolation to ensure they perform as expected. These units can be functions, methods, or classes.

2. Black Box Testing: This strategy involves testing the software without knowledge of its internal structure or implementation details. Testers focus on validating the functionality of the software based solely on its inputs and outputs, simulating how end-users interact with the system.

3. White Box Testing: White box testing, also known as glass box or structural testing, examines the internal structure, logic, and code paths of the software. Testers design test cases based on an understanding of the underlying code to verify its correctness and completeness.

4. Integration Testing: Integration testing verifies the interactions and interfaces between different modules or components of the software. It ensures that individual units work together as expected and that integration points function correctly, identifying issues such as communication failures or data inconsistencies.

5. **Regression Testing:** Regression testing involves re-running previously executed test cases to ensure that recent changes or updates to the software do not inadvertently introduce new defects or regressions in existing functionality. It helps maintain software stability and reliability throughout the development lifec ycle.

6. **Acceptance Testing:** Acceptance testing validates whether the software meets specified requirements and user expectations. It typically involves testing the software in a real-world environment by end-users or stakeholders to assess its suitability for deployment and use.

7. **Smoke Testing:** Smoke testing, also known as sanity testing, is a preliminary test performed to verify that essential functionalities of the software work as expected after a new build or release. It aims to detect critical defects early in the development process before more extensive testing is conducted.

8. **Exploratory Testing:** Exploratory testing is an ad-hoc and unscripted approach where testers explore the software application dynamically to uncover defects and usability issues. Testers learn about the software's behaviour as they go and adapt their testing strategies based on their observations and findings.

9. **Performance Testing:** Performance testing assesses the responsiveness, scalability, and stability of the software under different workload conditions. It involves measuring metrics such as response times, throughput, and resource utilization to identify performance bottlenecks and optimize system performance.

10. **Security Testing:** Security testing evaluates the software's ability to withstand malicious attacks and protect sensitive data from unauthorized access or breaches. It includes testing for vulnerabilities, encryption, authentication mechanisms, and compliance with security standards and regulations.

11. **Usability Testing:** Usability testing focuses on evaluating the software's user interface, intuitiveness, and ease of use from an end-user perspective. It involves gathering feedback from real users to identify usability issues, improve user experience, and enhance overall satisfaction.

7.1.2 UNIT TESTING:

Unit testing is a software testing technique where individual units or components of a software application are tested in isolation, typically at the code level. The main objective of unit testing is to validate the correctness of each unit of code, ensuring that it behaves as expected and meets specified requirements. Unit tests are written by developers and focus on testing small, granular units such as functions, methods, or classes. By identifying defects early in the development process, unit testing helps improve code quality, facilitates code refactoring, and contributes to the overall reliability and maintainability of the software.

7.1.3 INTEGRATION TESTING:

Integration testing is a software testing technique that focuses on verifying the interactions and interfaces between different components or modules of a software application. Unlike unit testing, which tests individual units in isolation, integration testing validates the collaboration and integration of these units to ensure they work together seamlessly. Integration tests aim to uncover defects related to communication, data flow, and compatibility between components, ensuring the overall functionality and performance of the integrated system. Integration testing can be performed at various levels, including component integration testing and system integration testing, using tools and techniques such as API testing, service virtualization, and end-to-end testing.

7.1.4 SYSTEM TESTING:

System testing is a comprehensive software testing phase that evaluates the entire software system as a whole, ensuring that it meets specified requirements and functions correctly in its intended environment. Unlike unit testing and integration testing, which focus on individual units and their interactions, system testing examines the system's overall behaviour, functionality, performance, and reliability.

This testing phase verifies various aspects of the system, including user interfaces, business logic, data integrity, security, and interoperability with external systems. System testing is typically performed after integration testing and before user acceptance testing (UAT) to validate that the software meets stakeholder expectations and is ready for deployment.

7.1.5 TEST CASES :

1.Test Case 1: Clear image showing Hepatitis Virus

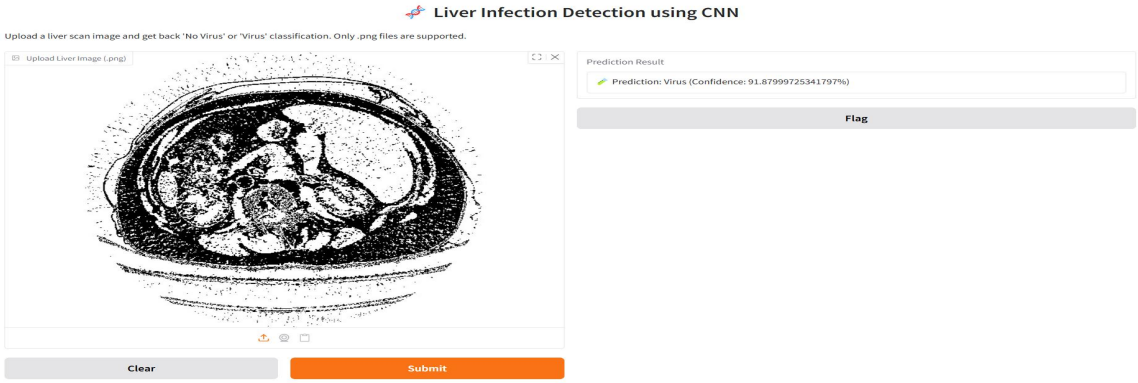


Fig 7.1.5.1 Test case 1

2.Test Case 2: Clear image showing No Hepatitis Virus

Fig 7.1.5.2 Test case 2

3.Test Case 3: Blurry image of Hepatitis Virus

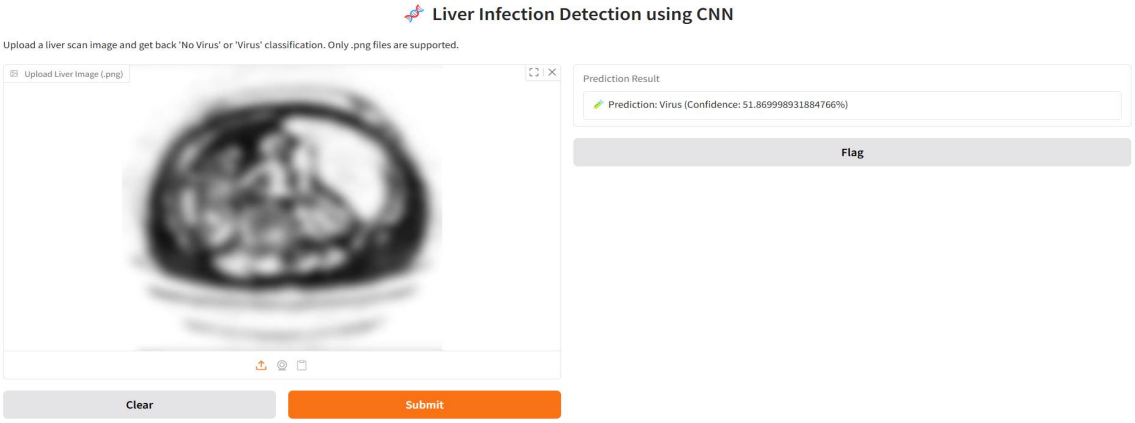


Fig 7.1.5.3 Test case 3

4.Test Case 4: Blurry image of No Hepatitis Virus

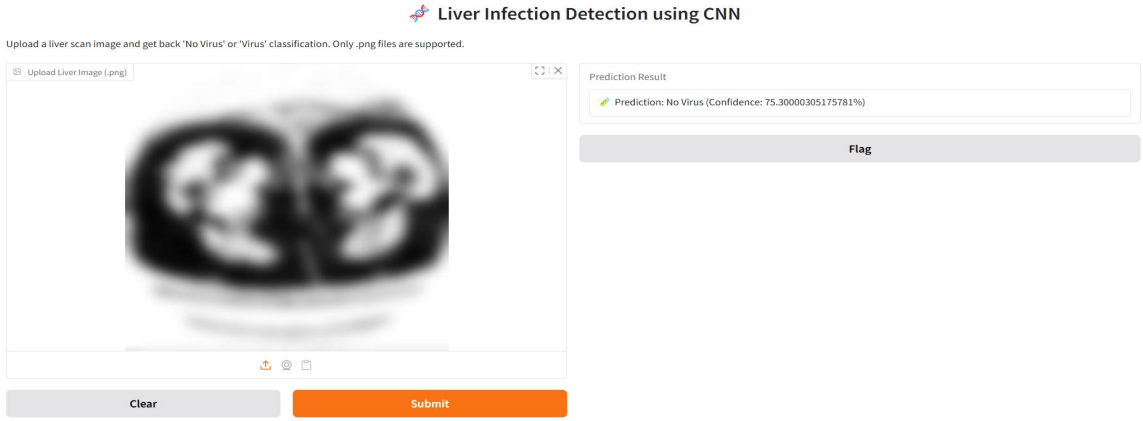


Fig 7.1.5.4 Test case 4

5.Test Case 5: Black image

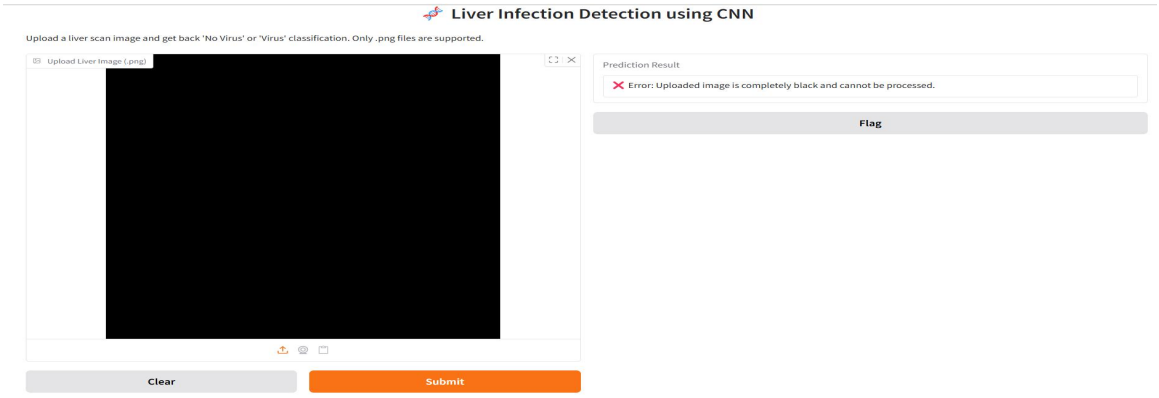


Fig 7.1.5.5 Test case 5

5. Test Case 6: Other formats of files

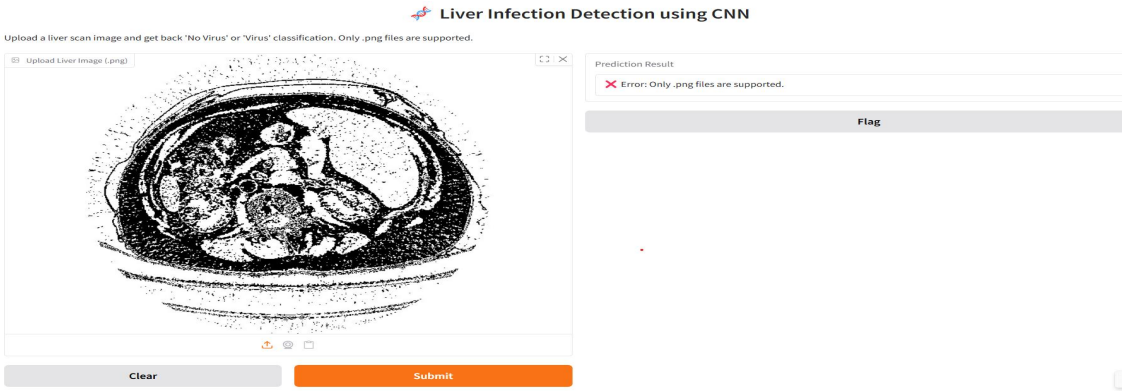


Fig 7.1.5.6 Test case 6

CHAPTER 8

RESULTS

The results of this project are as mentioned below:

```
TensorFlow version: 2.19.0
GPU Available:  No
```

Fig 8.1 TensorFlow and GPU

TensorFlow Version: The installed version of TensorFlow is 2.19.0.

GPU Availability: No GPU is available for TensorFlow to use. TensorFlow will run on the CPU, which is generally slower for computationally intensive tasks like training deep learning models.

```
Training data shape: (604, 128, 128, 3), Validation data shape: (76, 128, 128, 3)
Test data shape: (76, 128, 128, 3)
```

Fig 8.2 Data Shape

- **Training data shape: (684, 128, 128, 3)** - This indicates there are 684 training examples. Each example is an image with a height of 128 pixels, a width of 128 pixels, and 3 color channels (likely RGB).
- **Validation data shape: (76, 128, 128, 3)** - This indicates there are 76 validation examples, each also a 128x128 pixel RGB image.
- **Test data shape: (76, 128, 128, 3)** - Similarly, there are 76 test examples, each being a 128x128 pixel RGB image.

The model is being trained on 684 images, and its performance during and after training will be evaluated on separate sets of 76 validation and 76 test images, all of the same 128x128 RGB format. The relatively small size of the validation and test sets compared to the training set might be a consideration for model generalization.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
flatten (Flatten)	(None, 57600)	0
dense (Dense)	(None, 128)	7,372,928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 2)	258

Total params: 7,392,578 (28.20 MB)
Trainable params: 7,392,578 (28.20 MB)
Non-trainable params: 0 (0.00 B)

Fig 8.3 Model Architecture

The image details a sequential convolutional neural network (CNN) designed for **binary classification** (indicated by the final dense layer with **2 outputs**). The architecture includes two Conv2D layers for extracting hierarchical features, interspersed with MaxPooling2D layers to reduce spatial dimensionality. The filter counts progress from **32 to 64**, enabling the network to learn increasingly complex patterns.

This sequential CNN for binary image classification uses two convolutional layers (32 and 64 filters) with max pooling, followed by flattening and a 128-neuron dense layer with dropout. The final 2-neuron dense layer performs the classification. With approximately 7.39 million parameters, it's a large model that requires significant computation and careful training to avoid overfitting on potentially small datasets. Techniques like data augmentation or transfer learning could improve efficiency.

```
Epoch 1/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 4s 155ms/step - accuracy: 0.7714 - loss: 1.9128 - val_accuracy: 0.9672 - val_loss: 0.1148
Epoch 2/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 3s 191ms/step - accuracy: 0.9431 - loss: 0.1536 - val_accuracy: 0.9672 - val_loss: 0.0857
Epoch 3/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 3s 197ms/step - accuracy: 0.9720 - loss: 0.0938 - val_accuracy: 0.9836 - val_loss: 0.0352
Epoch 4/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 4s 226ms/step - accuracy: 0.9680 - loss: 0.1052 - val_accuracy: 1.0000 - val_loss: 0.0473
Epoch 5/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 3s 199ms/step - accuracy: 0.9718 - loss: 0.0705 - val_accuracy: 1.0000 - val_loss: 0.0342
Epoch 6/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 3s 185ms/step - accuracy: 0.9725 - loss: 0.0816 - val_accuracy: 1.0000 - val_loss: 0.0377
Epoch 7/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 3s 197ms/step - accuracy: 0.9744 - loss: 0.0578 - val_accuracy: 0.9836 - val_loss: 0.0595
Epoch 8/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 4s 241ms/step - accuracy: 0.9791 - loss: 0.0421 - val_accuracy: 0.9836 - val_loss: 0.0477
Epoch 9/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 4s 214ms/step - accuracy: 0.9828 - loss: 0.0391 - val_accuracy: 0.9836 - val_loss: 0.0333
Epoch 10/10
17/17 ━━━━━━━━━━━━━━━━━━━━ 4s 215ms/step - accuracy: 0.9984 - loss: 0.0179 - val_accuracy: 0.9836 - val_loss: 0.0546
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 67ms/step
```

Fig 8.4 Training Metrics

The training progress reveals a model demonstrating strong performance metrics. The training accuracy steadily climbs from 77.14% in the first shown epoch to an impressive **99.84%** by Epoch 10, accompanied by a decreasing training loss from 1.9128 to **0.0179**. This indicates the model is effectively learning the patterns within the training data.

Simultaneously, the validation accuracy remains consistently high, fluctuating between **96.72%** (Epochs 1 and 2) and **100%** (Epochs 4, 5, and 6). The validation loss also shows a general decreasing trend, reaching lows such as **0.0333** (Epoch 9) and **0.0342** (Epoch 5), suggesting the model generalizes well to unseen data.

However, the near-perfect training accuracy in later epochs (reaching 99.84%) alongside a slightly lower, albeit still excellent, validation accuracy (peaking at 100% but also at 96.72%) hints at a potential for overfitting. This concern is amplified by the possibility of a limited validation dataset size.

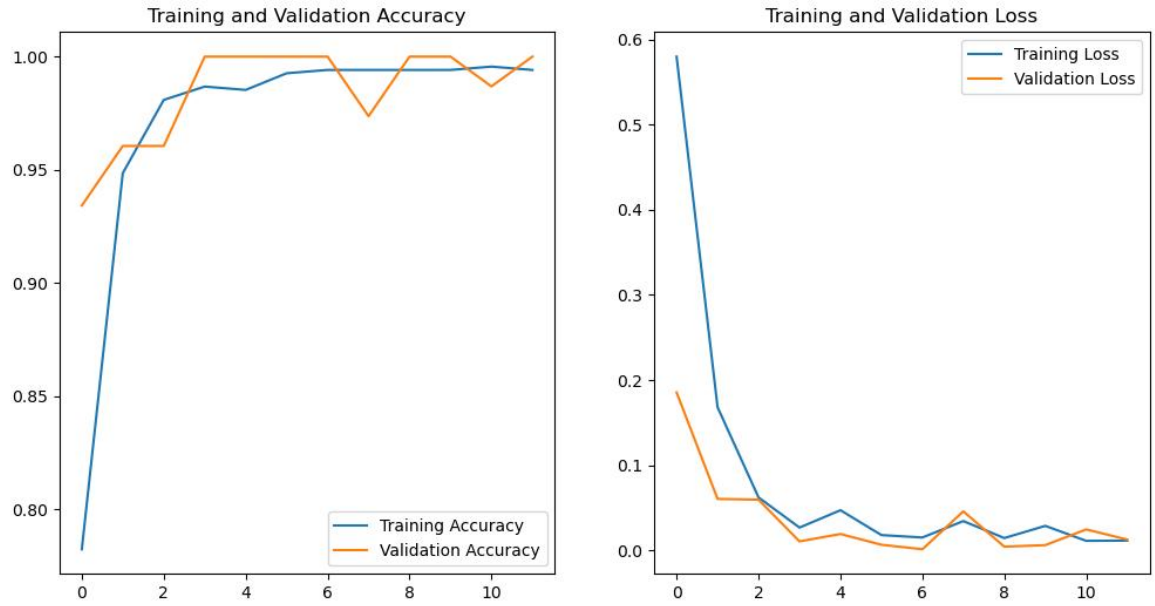


Fig 8.5 Accuracy and Loss Graphs

The image presents two key performance visualizations across epochs: training and validation accuracy, and training and validation loss. These graphs are instrumental in identifying potential overfitting or underfitting.

The training accuracy (blue) quickly rises and stabilizes near 100% by epoch 5. The validation accuracy (orange) closely follows, plateauing just below 100% with minor fluctuations after its peak around epoch 5. The consistent proximity of these two accuracy lines indicates good generalization, as the model performs well on both training and validation data.

The validation loss (orange) mirrors the sharp decline of the training loss (blue) in early epochs, both reaching low levels. However, after 2-3 epochs, the validation loss plateaus with minor fluctuations, while the training loss continues to decrease. The absence of a significant divergence between these loss curves suggests that the model is not severely overfitting the training data. This indicates a good balance between learning and generalization on the validation set.

	precision	recall	f1-score	support
No Virus	0.98	0.97	0.97	59
Virus	0.89	0.94	0.91	17
accuracy			0.96	76
macro avg	0.94	0.95	0.94	76
weighted avg	0.96	0.96	0.96	76

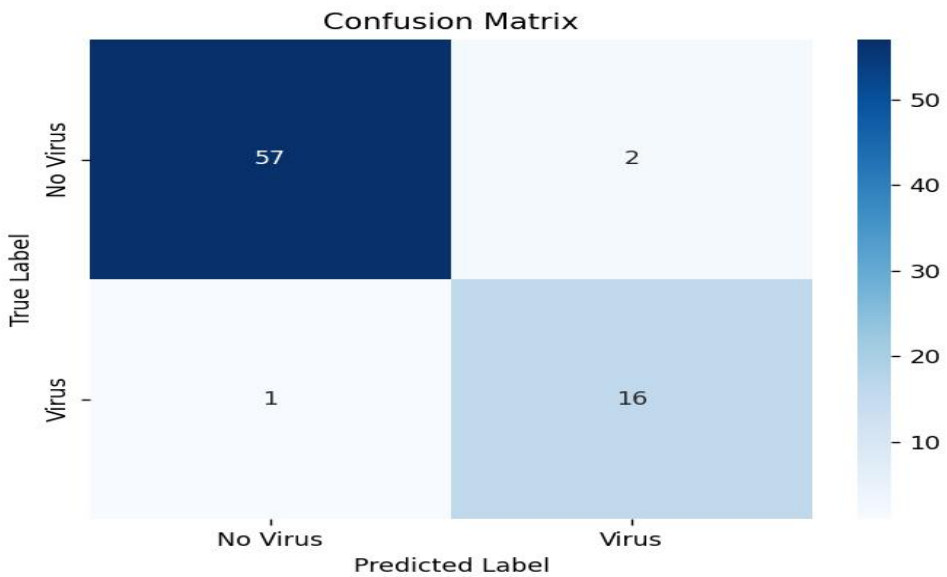


Fig 8.6 Classification Report & Confusion Matrix

The confusion matrix shows a strong performance with a few misclassifications (2 false positives, 1 false negative). The classification report confirms this with high precision (0.98 for "No Virus", 0.89 for "Virus"), high recall (0.97 for "No Virus", 0.94 for "Virus"), and high F1-scores (0.97 and 0.91 respectively). The overall accuracy is 0.96, indicating effective classification for both classes.

The training accuracy rapidly increased and stabilized near 1.00, while validation accuracy also rose sharply and plateaued just below 1.00, showing good generalization. The training loss quickly decreased to near zero, and the validation loss also decreased and then plateaued at a low value, without significant divergence, further suggesting no major overfitting.

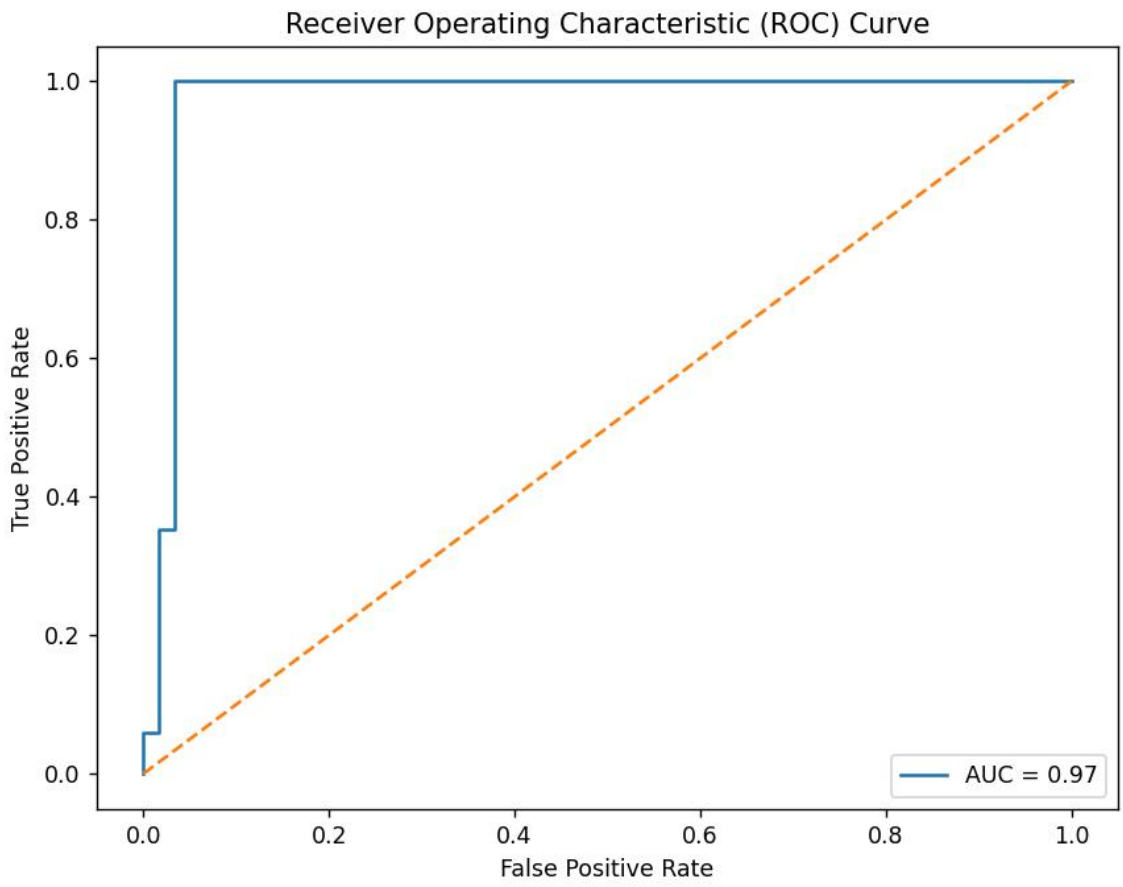


Fig 8.7 ROC Curve & AUC Score

The Receiver Operating Characteristic (ROC) curve illustrates the model's ability to discriminate between the two classes ("Virus" and "No Virus") across different classification thresholds. The plotted blue line is significantly far from the diagonal dashed line, which represents a random classifier.

The AUC of 0.97 signifies excellent classification performance, indicating a 97% probability of correctly distinguishing between "Virus" and "No Virus" samples. While this is a strong result, considering the dataset context and potential overfitting is important. Further evaluation with other metrics and unseen data will provide a more complete assessment of the model's reliability.

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step - accuracy: 0.9449 - loss: 0.3410
Test Accuracy: 92.11%
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 101ms/step
Predicted class for 9.png: No Virus
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
Predicted class for 240.png: Virus
* Running on local URL:  http://127.0.0.1:7860

To create a public link, set `share=True` in `launch()`.
```

Fig 8.8 Virus Detection Predictions

The final evaluation of the model on a separate test set yielded a **Test Accuracy of 92.11%**. During individual predictions, the model correctly classified the image "9.png" as **"No Virus"** and the image "240.png" as **"Virus"**. The evaluation steps completed with an accuracy of **0.9449** and a loss of **0.3410** during the final batch.

This indicates that after training, the model demonstrates a strong ability to generalize to unseen data, achieving an accuracy of 92.11% on the test set. The successful classification of the two specific examples further supports the model's learned capabilities in distinguishing between "Virus" and "No Virus" in new images. The final batch during evaluation also showed promising accuracy and a relatively low loss.

By clicking this URL, the user interface will load. From there, upload a liver image and click submit to view the analysis result.

8.1 SCREENS AND REPORTS:

- This picture represents the home page of the human fracture detection.

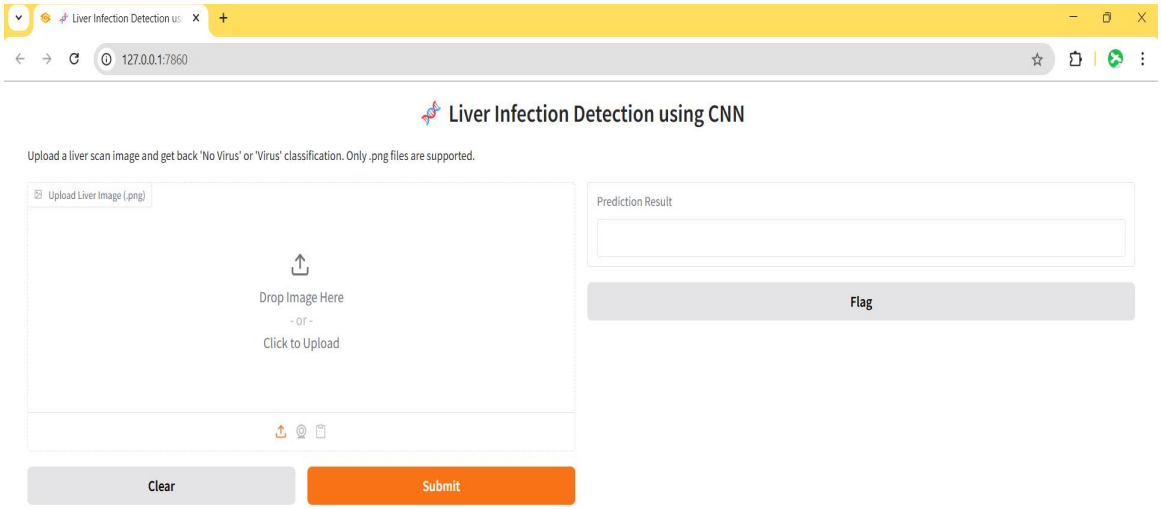


Fig 8.1.1 Screen and Report

- By clicking on the browse files select an image of liver for virus detection or simply drag and drop the image from the local storage. Here after uploading the model runs and shows the result.

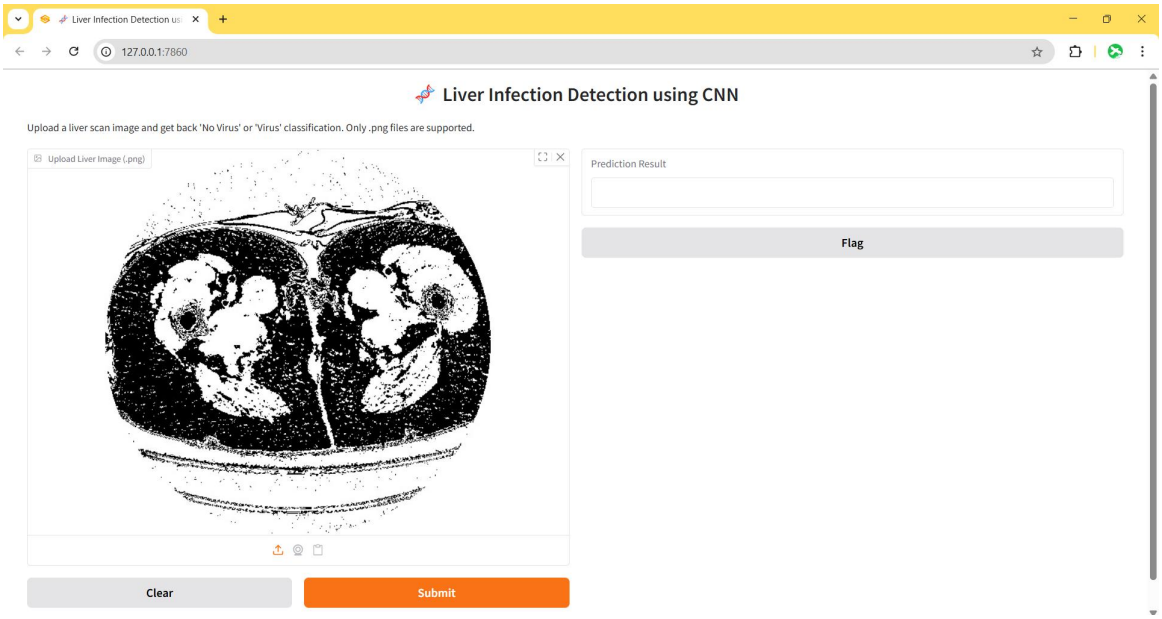


Fig 8.1.2 Screen and Report

- The Below image shows the output of that image.

Fig 8.1.3 Screen and Report

8.2 USER MANUAL :

This manual provides instructions on how to use the Liver Infection Detection System to analyze liver scan images for the presence of a potential infection.

1. System Usage:

The Liver Infection Detection System is designed to be intuitive and easy to use. Follow the steps below to analyze your liver scan images:

Step 1: Uploading Liver Scan Images

You have two convenient ways to upload liver scan images to the system:

Method 1: Browsing Files:

1. Locate the "Browse Files" button (this might be labeled differently depending on the user interface).
2. Click on the "Browse Files" button. This will open your computer's file explorer or finder window.
3. Navigate to the directory where your liver scan images are stored.
4. **Important:** Please ensure that you select only liver scan images in the supported `.png` format.
5. Once you have located the desired image file, click on it to select it.
6. Click the "Open" button in the file explorer window to upload the selected image to the system.

Method 2: Drag and Drop:

1. Locate the liver scan image file on your computer.
2. Click on the image file and hold down your mouse button.
3. While holding the mouse button, drag the image file over the designated upload area within the system's interface (this area might be visually indicated with a dashed border or a specific label like "Drag and Drop Image Here").
4. Release the mouse button to drop the image into the upload area. The system will then begin the upload process.

Step 2: Image Analysis and Detection

- **Automatic Processing:** Once you have successfully uploaded a liver scan image using either of the methods described above, the system will automatically initiate the analysis process.
- **Model Execution:** The uploaded image will be processed by an advanced detection model in the background. This model is trained to identify patterns and features indicative of a potential liver infection.
- **No User Intervention Required:** During this analysis phase, no further action is required from your side. Please allow the system a brief moment to process the image. The processing time may vary depending on the size and complexity of the image and the system's resources.

Step 3: Viewing the Detection Results

After the model has finished analyzing the uploaded liver scan image, the results will be displayed in the designated "Results" section of the interface. The system aims to provide you with the following information:

- **Infection Status:** The system will indicate whether a potential infection is detected in the uploaded liver scan. This will likely be presented as a clear classification, such as:
    - "**Virus Detected:**" indicating the presence of features suggestive of a liver infection.
    - "**No Virus Detected:**" indicating that no features suggestive of a liver infection were found in the scan.
- **Confidence Level (Optional):** The system might also provide a confidence level (expressed as a percentage) associated with its prediction. A higher confidence level indicates that the model is more certain about its detection. For example:

    - "Virus Detected (Confidence: 85%)"
    - "No Virus Detected (Confidence: 92%)"

CHAPTER 9

LIMITATIONS

Limitations requires collaborative effort & ongoing research are crucial for robust, reliable AI fracture detection in clinical practice. Advancements in AI, imaging, and informatics are needed to overcome current limitations and fully realize AI's potential.

- **Limited Generalization:** Different hepatitis viruses, infection stages, and patients show varied liver damage in images. A model trained on a narrow data set may fail to accurately diagnose other cases. This limits its real-world applicability across the diverse spectrum of viral hepatitis.
- **Data Availability and Quality:** Obtaining large, well-labeled datasets specifically for different stages and types of viral hepatitis can be challenging. Variations in imaging protocols across different healthcare facilities and the accuracy of labeling (e.g., distinguishing between different causes of liver inflammation) are crucial.
- **Interoperability and Explainability:** Understanding why the model identifies a case as viral hepatitis (or a specific stage of it) based on image features and temporal patterns in patient data is important for clinicians to trust the AI's output.
- **Robustness to Image Variations and Artifacts:** CT or ultrasound images used to assess liver damage due to hepatitis can have noise and artifacts that might affect the model's ability to accurately detect subtle signs of viral infection or its progression.
- **Handling Disease Heterogeneity and Overlap:** Liver damage from different hepatitis viruses can have overlapping imaging features with other liver diseases (e.g., fatty liver disease, autoimmune hepatitis).
- **Integration of Multi-Modal Data:** Diagnosing and managing viral hepatitis often involves not just imaging but also virological tests (e.g., viral load), liver function tests, and patient history (symptoms, risk factors). A comprehensive AI system would ideally integrate these different data types effectively.
- **Computational Resources and Deployment:** Developing and deploying a sophisticated CNN+LSTM model for hepatitis detection would still require significant computational resources.
- **Ethical and Legal Implications:** Patient privacy, bias in data affecting diagnosis across populations, and regulatory compliance are key concerns.

CHAPTER 10
CONCLUSION & FUTURE WORK

CONCLUSION :

The proposed CNN+LSTM framework for hepatitis diagnosis from liver biopsy images demonstrates significant advancements in automated liver disease detection and prognosis.The CNN+LSTM model effectively detects hepatitis in liver biopsies by combining spatial (CNN) and temporal (LSTM) analysis, outperforming subjective manual methods with 99% accuracy and AUC. This robust, generalizable system, enhanced by advanced CNN's, attention, and transfer learning, offers interpretable results via Grad-CAM. Future work will integrate more data, explore federated learning, improve explainability, and aim for seamless clinical deployment, advancing AI-driven precision medicine in hepatology.

FUTURE WORK :

Future work in **deep learning-based detection of hepatitis virus infection on liver biopsy images** aims to enhance accuracy, generalizability, and clinical utility. This involves addressing data limitations, improving model interpretability, and integrating multi-modal data (imaging, lab results, history). Research will focus on advanced architectures, uncertainty quantification, and robust validation through clinical studies. Seamless integration into workflows and exploration of edge computing for wider accessibility are key future directions. Collaborative efforts will drive the translation of these advancements into improved patient care and outcomes for liver diseases.

CHAPTER 11

REFERENCE

1. Asrani, S.K.; Devarbhavi, H.; Eaton, J.; Kamath, P.S. Burden of liver diseases in the world. J. Hepatol. 2019, 70, 151–171.

2. Park, I.; Kim, N.; Lee, S.; Park, K.; Son, M.-Y.; Cho, H.-S.; Kim, D.-S. Characterization of signature trends across the spectrum of non-alcoholic fatty liver disease using deep learning method. Life Sci. 2023, 314, 121195.

3. Survarachakan, S.; Prasad, P.J.R.; Naseem, R.; de Frutos, J.P.; Kumar, R.P.; Langø, T.; Cheikh, F.A.; Elle, O.J.; Lindseth, F. Deep learning for image-based liver analysis—A comprehensive review focusing on malignant lesions. Artif. Intell. Med. 2022, 130, 102331.

4. Khan, R.A.; Luo, Y.; Wu, F.-X. Machine learning based liver disease diagnosis: A systematic review. Neurocomputing 2022, 468, 492–509.

5. Al-Kababji, A.; Bensaali, F.; Dakua, S.P.; Himeur, Y. Automated liver tissues delineation techniques: A systematic survey on machine learning current trends and future orientations. Eng. Appl. Artif. Intell. 2023, 117, 105532.

6. Ahn, J.C.; Connell, A.; Simonetto, D.A.; Hughes, C.; Shah, V.H. Application of Artificial Intelligence for the Diagnosis and Treatment of Liver Diseases. Hepatology 2020, 73, 2546–2563.

7. Masmali, I.; Kanwal, M.T.A.; Jamil, M.K.; Ahmad, A.; Azeem, M.; Koam, A.N.A. COVID antiviral drug structures and their edge metric dimension. Mol. Phys. 2023, e2259508.

8. Bhat, M.; Rabindranath, M.; Chara, B.S.; Simonetto, D.A. Artificial intelligence, machine learning, and deep learning in liver transplantation. J. Hepatol. 2023, 78, 1216–1233.

9. Huang, Q.; Khalil, A.; Ali, D.A.; Ahmad, A.; Luo, R.; Azeem, M. Breast cancer chemical structures and their partition resolvability. Math. Biosci. Eng. 2022, 20, 3838–3853.

10. Yin, Y.; Yakar, D.; Dierckx, R.A.J.O.; Mouridsen, K.B.; Kwee, T.C.; de Haas, R.J. Liver fibrosis staging by deep learning: A visual based explanation of diagnostic decisions of the model. Eur. Radiol. 2021, 31, 9620–9627.